

PROJEKARBEIT

„Edge AI Computing mit TVM auf einem STM32 Mikrocontroller“

YVAN VALDER SIMO GUENO

betreut von

Prof. Dr. -Ing. Michael Athanassios Karagounis

Wintersemester 2023

Abstract

In diesem Bericht wird die Rolle der Künstlichen Intelligenz (KI) in der modernen Gesellschaft untersucht und ihr breites Spektrum an Anwendungen und möglichen zukünftigen Richtungen aufgezeigt. Dank ihrer Fähigkeit, menschenähnliches Verhalten zu reproduzieren, bietet KI eine breite Palette von Anwendungen in verschiedenen Branchen. Apache TVM ist eine Open-Source-Software, die maschinelles Lernen auf verschiedenen Plattformen, einschließlich Edge-Computing-Geräten, ermöglicht, um Daten in Echtzeit zu analysieren. Diese Projektarbeit demonstriert Bilderkennung als praktische Anwendung von KI unter Verwendung des NUCLEO L4R5ZI Boards, wobei die Integration mit einer benutzerdefinierten Entwicklungsumgebung gezeigt wird.

This Thesis examines the role of artificial intelligence (AI) in modern society and highlights its wide range of applications and possible future directions. Thanks to its ability to reproduce human-like behaviour, AI offers a wide range of applications in various industries. Apache TVM is an open-source software that enables machine learning on various platforms, including edge computing devices, to analyse data in real time. This report demonstrates image recognition as a practical application of AI using the NUCLEO L4R5ZI board, showing integration with a custom development environment. In summary, the pervasive impact of AI highlights its importance in shaping future progress and emphasises the need to adapt to societal values.

Inhaltverzeichnis

Abstract	2
Inhaltverzeichnis	3
Einleitung.....	4
1. Maschinelles Lernen	5
1.1 Was ist Machine Learning?	5
1.2 Warum wird Maschine Learning verwendet?	5
1.3 Unterschiedliche Machine-Learning-Systeme	6
I. Überwachtes/unüberwachtes Lernen	6
II. Batch- und Online-Learning.....	8
III. Instanz basiertes versus modellbasiertes Lernen	8
1.4 Herausforderungen beim Machine-Learning	8
2. Neuronale Netze und Deep Learning.....	9
2.1. Einführung in künstlichen neuronalen Netzen (KNN).....	9
a) Anwendungsgebiete Neuronale Netze	10
b) Arten von neuronalen Netzen	10
3. Deep Learning.....	11
3.1 Trainieren von Deep-Learning-Netzen (bzw. neuronaler Netze).....	12
4. Trainieren von Modellen.....	12
5. Integration von microTVM in die Entwicklungsumgebung.....	13
5.1 Projekt: custom microTVM in GoogleColab.....	14
5.1.1. Apache TVM (bzw. microTVM).....	14
5.1.2. Notwendige Python-Pakete.....	15
5.1.3. TFLite Model importieren	16
5.1.4. Generierung des Model Library Format (MLF).....	17
5.1.5. CMSIS-Bibliothek	18
5.1.6. Modellevaluation	19
5.1.7. Modell integrieren	20
5.2 Projekt: microTVM in STM32CubeIDE	21
5.3 Erstellung einer GUI zur Auswertung von Bildern mit QT- Creator.....	32
5.4 Auswertung von Bildern.....	46
Fazit.....	49
Quelle und Literaturverzeichnis.....	50
Abbildungsverzeichnis.....	50

Einleitung

Künstliche Intelligenz (KI) hat in den letzten Jahren zunehmend an Bedeutung gewonnen und bietet ein großes Potenzial für die Zukunft. Die Vorstellung, dass Maschinen wie Menschen denken, lernen und handeln können, ist sehr interessant und bietet viele Anwendungsmöglichkeiten. Künstliche Intelligenz hat verschiedene Definitionen und bezieht sich darauf, wie Computersysteme auf der Grundlage von Vernunft, Denken und Handeln arbeiten.

Apache TVM ist eine Open-Source-Software, die auf Aufgaben des maschinellen Lernens spezialisiert ist. Sie ermöglicht die Optimierung und Ausführung von Modellen auf einer Vielzahl von Plattformen und Geräten, einschließlich Edge-Computing-Geräten. Durch die Kombination von Edge Computing und maschinellem Lernen können Echtzeitdaten auf Edge-Geräten analysiert und für Echtzeitentscheidungen genutzt werden, ohne auf eine Cloud-Infrastruktur angewiesen zu sein. Diese Fähigkeiten machen Apache TVM und Edge Computing zu wichtigen Technologien für die Entwicklung intelligenter Systeme in Bereichen wie Industrieautomatisierung, Automotive und Robotik.

1. Maschinelles Lernen

1.1 Was ist Machine Learning?

Machine Learning (ML) ist die Wissenschaft (sowie die Kunst), Computer so zu programmieren, dass sie anhand von Daten lernen. Das heißt dieses Teilgebiet der Künstlichen Intelligenz verleiht dem Computer die Fähigkeit zu lernen, ohne explizit programmiert zu werden.

Allerdings ist Machine Learning nicht so futuristisch, wie allgemein angenommen. Bereits in der Vergangenheit waren Spamfilter, die erste weitverbreitete Anwendung von ML, die das Leben von sehr vielen Menschen verbessert hat. Spamfilter sind in der Tat maschinelle Lernprogramme, die aus vom Nutzer markierten Beispielen (Lehrbeispiel oder Trainingsdatensatz) für Spam-E-Mails („Junks-Mail“) und gewöhnlichen Emails lernt. Dabei wird jedes Trainingsbeispiel als Trainingsdatenpunkt (oder Instanz) bezeichnet.

1.2 Warum wird Maschine Learning verwendet?

Maschinelles Lernen ist besonders nützlich für Aufgaben, die viel Feinarbeit oder lange Listen von Regeln erfordern, in diesen Fällen wird durch ML-Algorithmen der Code oft vereinfacht und zeitgleich bessere Ergebnisse erzielt als mit herkömmlichen Ansätzen. ML ist auch geeignet für komplexe Aufgaben ohne klare Lösungen, da ML-Techniken möglicherweise eine Lösung finden können. In fluktuierenden Umgebungen kann sich ein ML-System an neue Daten anpassen. Darüber hinaus ermöglicht ML, Erkenntnisse aus komplexen Aufgabenstellungen und großen Datenmengen zu gewinnen, indem es Muster erkennt und wertvolle Informationen extrahiert. Insgesamt bietet maschinelles Lernen einen leistungsfähigen Ansatz für eine Vielzahl von Herausforderungen in verschiedenen Branchen und Bereichen. Ein Anwendungsbeispiel sind die Erkennung von Tumoren in Gehirnscans, die automatische Klassifikation von Nachrichtenartikeln, die automatische Markierung von beleidigenden Kommentaren in Diskussionsforen, die Erstellung eines Chatbots oder persönlichen Assistenten, die Vorhersage des Umsatzes einer Firma für das nächste Jahr basierend auf vielen Performance Metriken usw.

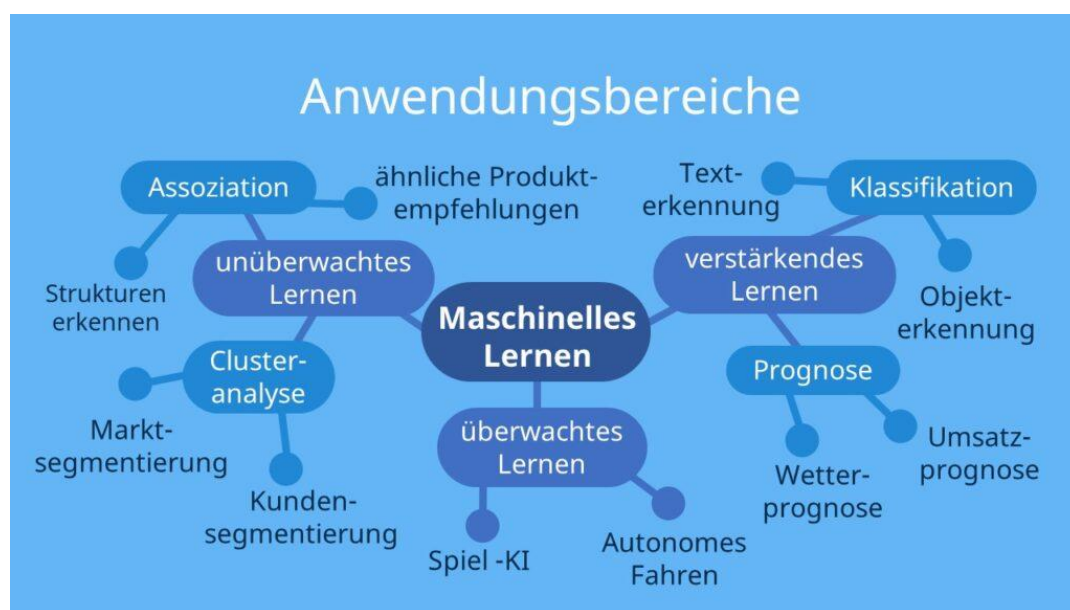


Abbildung 1: Machine Learning Anwendungsbereiche [1]

1.3 Unterschiedliche Machine-Learning-Systeme

Beim Maschinellen Lernen werden die folgenden Verfahren unterschieden.

I. Überwachtes/unüberwachtes Lernen

- **Überwachtes Lernen** (*Supervised Learning*):

Beim überwachten Lernen müssen Beispielmuster vordefiniert und spezifiziert werden, um die Informationen mit dem Mustersatz des Algorithmus abzugleichen.

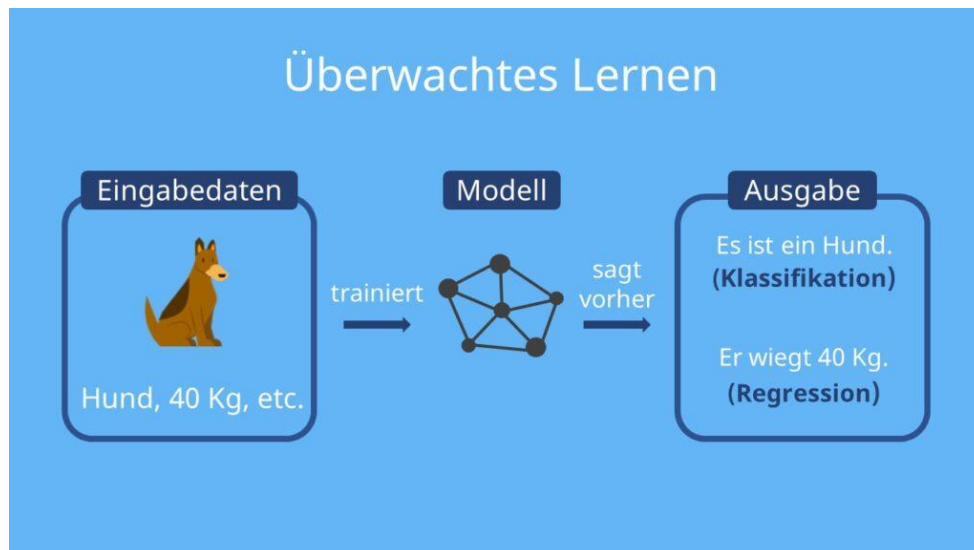


Abbildung 2: Ein Bild von Hund mit dem richtigen Ergebnis eingeben (Klassifikation= Hund und Regression= 40 in kg) [1]

- **Unüberwachtes Lernen** (*Unsupervised Machine Learning*):

Beim unüberwachten Lernen wird der Mustersatz automatisch aus unabhängig erkannten Mustern aufgebaut.

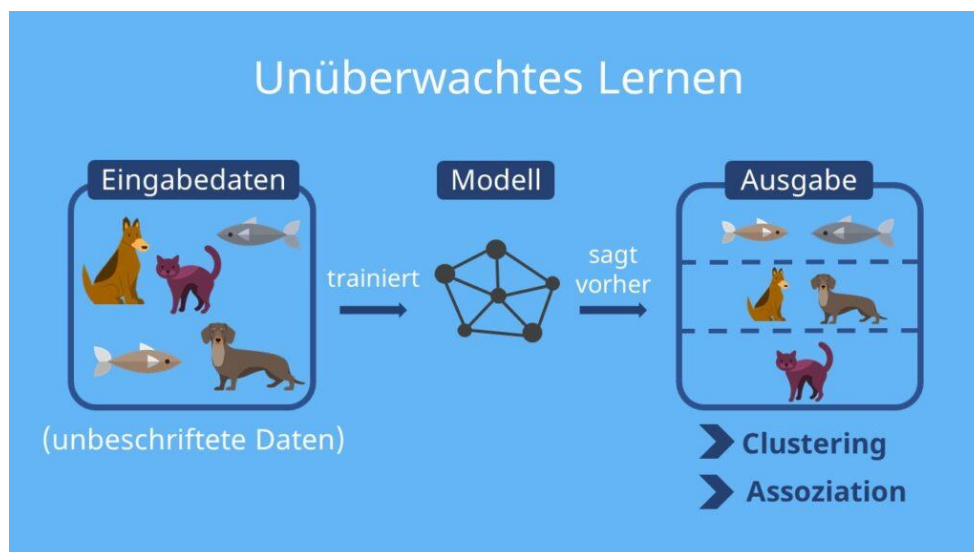


Abbildung 3: nur verschiedene Tierbilder, ohne Beschriftung (Clustering = Gruppe von Fisch, Gruppe von Hunden... Assoziation = Wer wählt Fisch, wählt auch Katze) [1]"

- **Semi-überwachtes Lernen (Halbüberwachtes Lernen):**
Semi-überwachtes Lernen ist eine Mischung aus den oben genannten beiden Methoden.

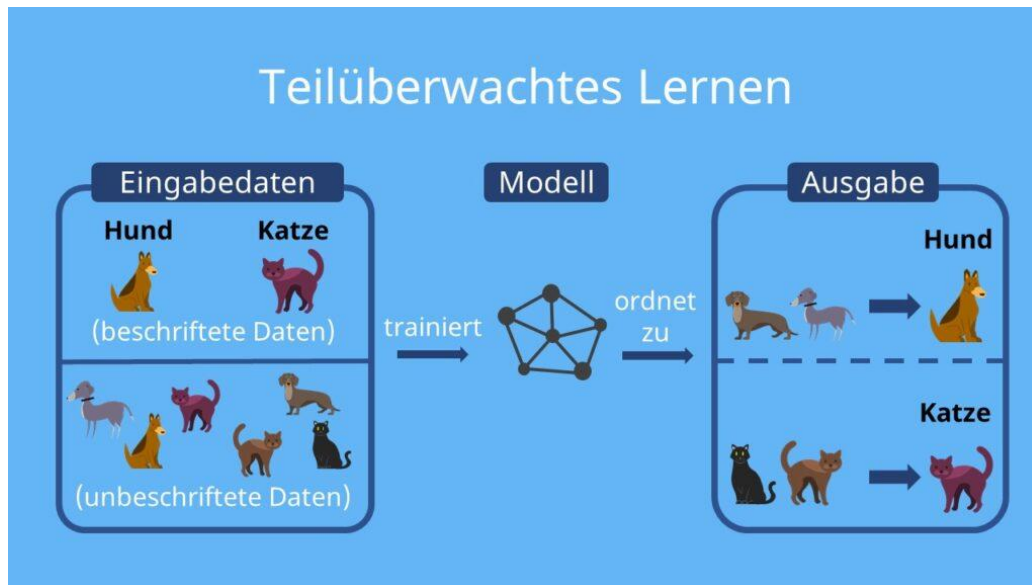


Abbildung 4: das manuelle Beschriften von Bildern mit der passenden Tierart [1]

- **Verstärkungslernen (Reinforcement Learning):**
Das Verstärkungslernen basiert auf Belohnungs- und Bestrafungsprinzip. Diese Interaktion bestimmt, wie der Algorithmus auf verschiedene Situationen reagiert. Diese Art des Lernens ist dem menschlichen Lernen sehr ähnlich, denn das System muss selbst herausfinden, was die beste Strategie oder Policy ist, um mit der Zeit die meisten Belohnungen zu erhalten. Eine Policy definiert, welche Aktion des Lernsystem (Agent) in einer gegebenen Situation auswählt.



Abbildung 5: verstärkendes Lernen [1]

II. Batch- und Online-Learning

- **Batch/Offline-Learning:**

Beim Batch-Learning kann das System nicht inkrementell lernen, es muss mit sämtlichen verfügbaren Daten trainiert werden. Dies dauert meist lange, beansprucht Rechenkapazitäten und wird daher in der Regel offline durchgeführt.

- **Online-Learning:**

Beim Online-Learning wird das System nach und nach trainiert, indem einzelne Datensätze nacheinander oder in kleinen Paketen, sogenannten Mini-Batches, hinzugefügt werden. Jeder Lernschritt ist schnell und billig, sodass das System aus neuen Daten lernen kann, sobald diese verfügbar sind

III. Instanz basiertes versus modellbasiertes Lernen

- Beim **Instanz basierten Lernen** werden Trainingsdaten direkt im Speicher gespeichert und für Vorhersagen genutzt, indem ähnliche Datenpunkte aus dem Trainingsdatensatz verwendet werden.
- Beim **modellbasierten Lernen** werden Trainingsdaten verwendet, um ein explizites Modell zu erstellen, das die Beziehung zwischen Eingabe- und Ausgabewerten beschreibt, welches dann zur Vorhersage neuer Daten verwendet wird.

1.4 Herausforderungen beim Machine-Learning

Datenqualität und -quantität: Die Qualität und Quantität der verfügbaren Daten sind entscheidend für die Leistungsfähigkeit von ML-Modellen. Minderwertige oder unzureichende Daten können zu fehlerhaften Modellen und Vorhersagen führen.

Overfitting und Underfitting: Overfitting tritt auf, wenn ein Modell zu stark an die Trainingsdaten angepasst ist und daher nicht in der Lage ist, auf neuen Daten gut zu generalisieren. Underfitting hingegen tritt auf, wenn ein Modell zu einfach ist und die Komplexität der Daten nicht richtig erfasst.

Feature Engineering: Die Auswahl und Konstruktion relevanter Merkmale aus den Rohdaten können eine komplexe und zeitaufwändige Aufgabe sein. Das richtige Feature Engineering ist jedoch entscheidend für die Leistungsfähigkeit eines ML-Modells.

Repräsentativität der Daten: Die Trainingsdaten sollten repräsentativ für die tatsächlichen Bedingungen sein, auf die das Modell angewendet wird. Wenn die Trainingsdaten nicht repräsentativ sind, können sich Verzerrungen und Ungenauigkeiten in den Vorhersagen des Modells einschleichen.

Interpretierbarkeit von Modellen: Bei komplexen ML-Modellen wie neuronalen Netzen kann die Interpretation der Ergebnisse schwierig sein. Es ist wichtig, dass ML-Modelle transparent und interpretierbar sind, insbesondere in sicherheitskritischen Anwendungen.

Skalierbarkeit und Effizienz: Die Skalierung von ML-Modellen auf große Datensätze oder komplexe Probleme kann herausfordernd sein. Die Effizienz von ML-Algorithmen und -Modellen sollte daher berücksichtigt werden, insbesondere in ressourcenbeschränkten Umgebungen.

Ethik und Fairness: ML-Modelle können durch Vorurteile in den Trainingsdaten oder durch die Optimierung bestimmter Zielfunktionen zu unfairen oder diskriminierenden Ergebnissen führen. Es ist wichtig, ethische Fragen im Zusammenhang mit dem Einsatz von ML-Technologien zu berücksichtigen und sicherzustellen, dass Modelle fair und gerecht sind.

2. Neuronale Netze und Deep Learning

2.1. Einführung in künstlichen neuronalen Netzen (KNN)

Ein KNN ist ein Machine-Learning-Modell, das von den Netzwerken der Neuronen in biologischen Gehirnen inspiriert ist. KNNs sind die Kernkomponente des Deep Learning und bestehen aus verschiedenen Knoten (Neuronen).

In einem neuronalen Netz erhält jedes Neuron gewichtete Eingaben von anderen Neuronen, verarbeitet sie und gibt eine Ausgabe aus. Die Verbindungen zwischen den Neuronen werden gewichtet, um die Bedeutung der einzelnen Eingaben zu differenzieren. Ein neuronales Netz besteht aus mehreren Schichten von Neuronen und wird normalerweise trainiert, um seine Aufgabe besser zu erledigen.

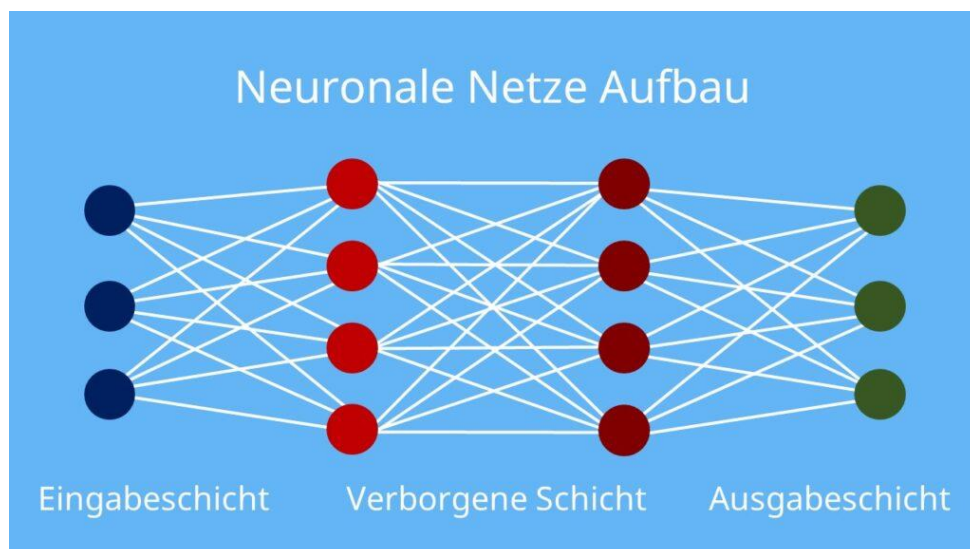


Abbildung 6: Aufbau Neuronaler Netze [1]

Eingabeschicht: Hier werden die Eingabedaten in das neuronale Netzwerk eingespeist. Jedes Eingabeneuron verarbeitet einen Teil der Eingabedaten und leitet diese gewichtet an die nächste Schicht weiter.

Verborgene Schicht: Diese Schicht befindet sich zwischen der Eingabeschicht und der Ausgabeschicht. Sie besteht aus mehreren Ebenen von Neuronen, welche die empfangenen Informationen erneut gewichten und von Neuron zu Neuron bis zur Ausgabeschicht weiterleiten. Die genaue Verarbeitung der Informationen in dieser Schicht ist nicht direkt sichtbar, woraus der Name "verborgene Schicht" folgt.

Ausgabeschicht: Dies ist die letzte Schicht des neuronalen Netzwerks, die unmittelbar an die letzte Ebene der verborgenen Schicht anschließt. Die Output-Neuronen in dieser Schicht enthalten die resultierenden Entscheidungen oder Vorhersagen des Netzwerks.

a) Anwendungsgebiete Neuronale Netze

Hauptsächlich eignen Neuronale Netze sich für Anwendungen, in denen man eine große Menge an unstrukturierten Daten verarbeiten und verknüpfen muss, um ein Ergebnis zu erhalten.

Anwendungsgebiete für Neuronale Netze sind:

- Bilderkennung
- Spracherkennung
- Erzeugen von Sprache
- autonomes Fahren
- Wettervorhersagen
- Frühwarnsysteme (Raumfahrt, Verteidigung, Erdbeben)
- Medizinische Systeme (Krankheitsanalyse)
- Betrugserkennung
- Vorhersage von Kundenverhalten
- Wirtschaftsmodelle

b) Arten von neuronalen Netzen

Perzeptron: Das Perzeptron ist das einfachste neuronale Netzwerk, bestehend aus einem einzelnen künstlichen Neuron. Es verarbeitet Eingabedaten und gibt basierend auf einer Schwellenwertfunktion eine binäre Ausgabe.

Feed Forward Neural Networks: Diese Netzwerke haben keine rückwärtigen Verbindungen zwischen den Schichten, wodurch eine klare Richtung des Informationsflusses entsteht. Sie bestehen aus Eingabe-, versteckten und Ausgabeschichten und werden häufig für verschiedene Aufgaben des maschinellen Lernens eingesetzt.

Recurrent Neural Networks (RNNs): Im Gegensatz zu Feed Forward Networks haben RNNs rückwärtsgerichtete Verbindungen, die es ihnen ermöglichen, Informationen über einen längeren Zeitraum zu behalten und auf vergangene Zustände zurückzugreifen. Sie werden häufig für die Verarbeitung von Sequenzdaten wie Texten oder Zeitreihen verwendet.

Convolutional Neural Networks (CNNs): CNNs sind besonders gut für die Verarbeitung und Klassifizierung von Bildern geeignet. Sie bestehen aus Schichten von Faltungsoperationen, die dazu dienen, wichtige Merkmale aus den Bildern zu extrahieren und sie dann durch eine Reihe von Schichten zu klassifizieren.

3. Deep Learning

Deep Learning ist eine spezielle Methode der Informationsverarbeitung und ein Teilbereich von Machine Learning und Künstlicher Intelligenz. Es verwendet neuronale Netzwerke zur Analyse großer Datensätze und agiert dabei ähnlich wie das menschliche Gehirn. Dabei werden Daten extrahiert, analysiert und anschließend Schlussfolgerungen oder Prognosen erstellt. Deep Learning ist besonders leistungsfähig bei der Verarbeitung unstrukturierter Daten wie Bildern, Texten und Sprache, da es automatisch relevante Merkmale aus den Daten extrahieren kann, ohne dass manuelle Merkmalskonstruktion erforderlich ist.

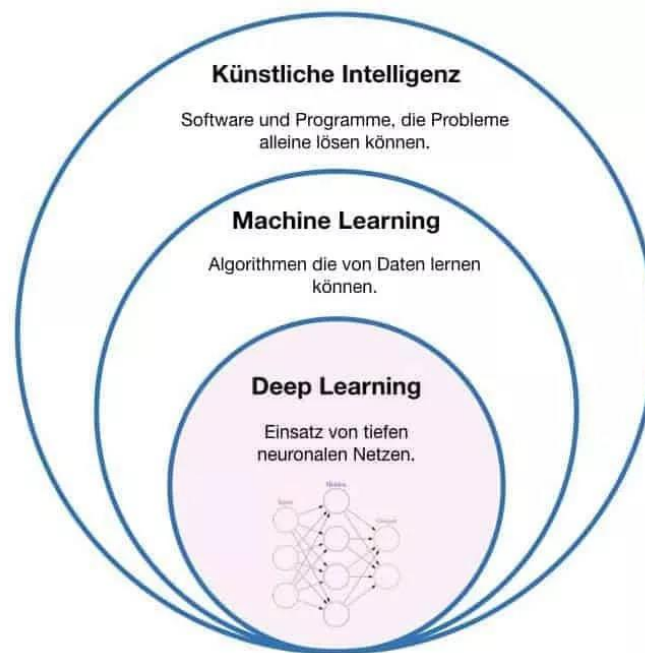


Abbildung 7: Deep Learning, Teil der künstlichen Intelligenz [2]

Deep Learning und klassisches Machine Learning sind beide Teilbereiche des maschinellen Lernens, jedoch unterscheiden sie sich in ihrer Herangehensweise und den verwendeten Techniken.

Klassisches Machine Learning umfasst eine Vielzahl von Algorithmen und Techniken, die darauf abzielen, Muster und Beziehungen in Daten zu identifizieren, um Vorhersagen oder Entscheidungen zu treffen. Diese Methoden können beispielsweise lineare Regression, logistische Regression, Entscheidungsbäume, Support Vector Machines und Clustering-Algorithmen wie k-Means umfassen. Im klassischen Machine Learning werden in der Regel manuell konstruierte Merkmale verwendet, um die Eingabedaten zu repräsentieren.

Im Gegensatz dazu konzentriert sich Deep Learning ausschließlich auf die Verwendung Künstlicher Neuronaler Netzwerke (KNN) für die Datenverarbeitung und -repräsentation. Diese Netzwerke bestehen aus vielen Schichten von Neuronen, die Informationen durch gewichtete Verbindungen verarbeiten.

Der Hauptunterschied liegt also darin, dass Deep Learning ausschließlich auf künstlichen neuronalen Netzwerken basiert, während klassisches Machine Learning eine Vielzahl von Methoden umfasst, die nicht unbedingt auf neuronalen Netzwerken beruhen.

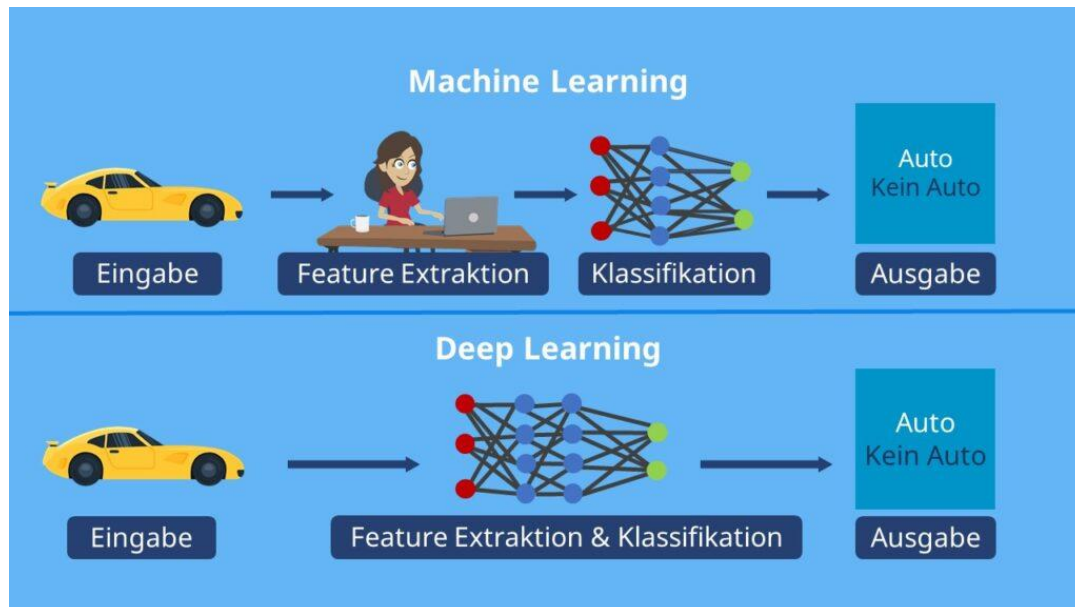


Abbildung 8: Machine Learning vs Deep Learning [3]

3.1 Trainieren von Deep-Learning-Netzen (bzw. neuronaler Netze)

Um ein künstliches neuronales Netzwerk zu trainieren und zu verbessern, werden kontinuierlich die Parameter justiert und optimiert. Dieser Prozess umfasst die Anpassung der Gewichtungen zwischen den Neuronen, insbesondere ausgehend von der Ausgabeschicht, durch komplexe mathematische Verfahren. Nach jeder Anpassung wird das Netzwerk erneut mit Daten gefüttert, und der Zyklus beginnt von vorne. Dieser Vorgang wird wiederholt, bis das neuronale Netzwerk die Daten korrekt klassifiziert oder vorhersagt. Letztendlich wird, die Genauigkeit und Leistungsfähigkeit des neuronalen Netzwerks maximiert, indem die Parameter so optimiert werden, dass das Netzwerk eine optimale Leistung zeigt.

4. Trainieren von Modellen

Modelle im maschinellen Lernen werden durch einen Prozess namens Training optimiert. Dieser Prozess beinhaltet typischerweise die folgenden Schritte:

Datenvorbereitung: Zuerst werden die Trainingsdaten vorbereitet. Dies umfasst die Reinigung der Daten, das Aufteilen in Trainings- und Validierungssets und die mögliche Transformation der Daten, um sie für das Modelltraining geeignet zu machen.

Auswahl des Modells: Je nach Art des Problems und den Eigenschaften der Daten wird ein geeignetes Modell ausgewählt. Dies kann ein lineares Modell, ein Entscheidungsbaum, ein neuronales Netzwerk oder eine andere Art von Modell sein.

Initialisierung der Parameter: Die Parameter des Modells werden initialisiert. Je nach Modelltyp können dies beispielsweise die Gewichte in einem neuronalen Netzwerk oder die Koeffizienten in einer linearen Regression sein.

Trainingsschleife: In einer Schleife werden die Trainingsdaten durch das Modell geschickt, und die Vorhersagen des Modells werden mit den tatsächlichen Werten verglichen. Basierend auf diesen Vergleichen werden die Parameter des Modells schrittweise angepasst, um die Vorhersagegenauigkeit zu verbessern.

Optimierungsalgorithmus: Ein Optimierungsalgorithmus wird verwendet, um die Parameter des Modells zu aktualisieren. Dies kann beispielsweise der Gradientenabstiegsalgorithmus sein, der den Fehler zwischen den Vorhersagen des Modells und den tatsächlichen Werten minimiert.

Validierung und Feinabstimmung: Das trainierte Modell wird auf einem separaten Validierungsdatensatz getestet, um sicherzustellen, dass es gut generalisiert und nicht überangepasst ist. Basierend auf den Validierungsergebnissen können Anpassungen am Modell vorgenommen werden, um die Leistung weiter zu verbessern.

Evaluation: Schließlich wird das trainierte Modell auf einem Testdatensatz evaluiert, um seine Leistung zu bewerten. Dies kann durch Metriken wie Genauigkeit geschehen.

5. Integration von microTVM in die Entwicklungsumgebung

Dieses Tutorial beschreibt die Schritte zur Integration eines bereits kompilierten Modells in eine eigene Entwicklungsumgebung. Es verwendet das Board NUCLEO L4R5ZI und die Entwicklungsumgebung STM32CubeIDE, um das Visual Wake Word (VWW) Modell von MLPerf Tiny mit microTVM zu integrieren. Obwohl spezifische Tools wie STM32CubeIDE und das NUCLEO L4R5ZI Board verwendet werden, können die gleichen Schritte auch auf andere Entwicklungsumgebungen und Mikrocontroller angewendet werden. Das Tutorial bietet eine Anleitung für die Integration von microTVM in benutzerdefinierte Entwicklungsumgebungen und die Verwendung von Modellen auf verschiedenen Zielplattformen.

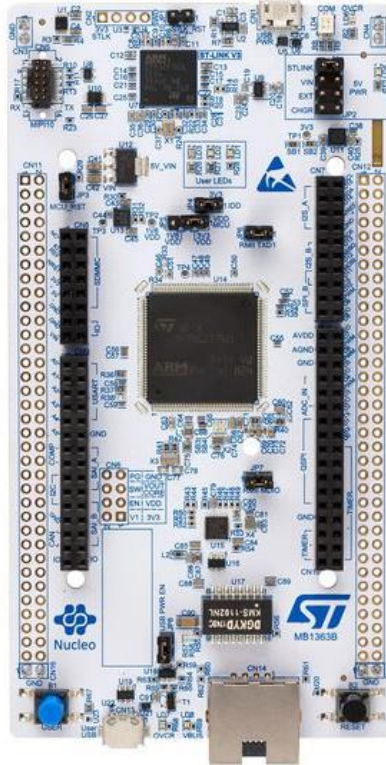


Abbildung 9: NUCLEO-L4R5ZI - STM32 Nucleo Board 2MB 640KB

5.1 Projekt: custom microTVM in GoogleColab

Dieses Tutorial wird mit Google Colab durchgeführt, einem Cloud-basierten Python-Codierungstool ähnlich einem Jupyter Notebook. Es ermöglicht Python-Programmierern, Code über einen Webbrowser zu schreiben und auszuführen, ohne dass eine lokale Installation erforderlich ist.

Wenn Sie Ihr Modell auf ein mobiles Gerät oder ein Embedded-Gerät bereitstellen möchten, kann die Verwendung eines großen Modells zu Problemen führen. Dazu gehören lange Downloadzeiten, hohe RAM- und CPU-Auslastung und eine beeinträchtigte App-Performance. Colab bietet einen Zugang zu einer Vielzahl von Ressourcen wie Arbeitsspeicher, Rechenkapazität, GPUs und TPUs, was besonders vorteilhaft für rechenintensive Anwendungen ist.

Google hat Colab speziell für die Bedürfnisse im Bereich maschinelles Lernen entwickelt, um Big Data-Analysten, Datenwissenschaftlern, KI-Forschern und Python-Lernende zu unterstützen.

5.1.1. Apache TVM (bzw. microTVM)

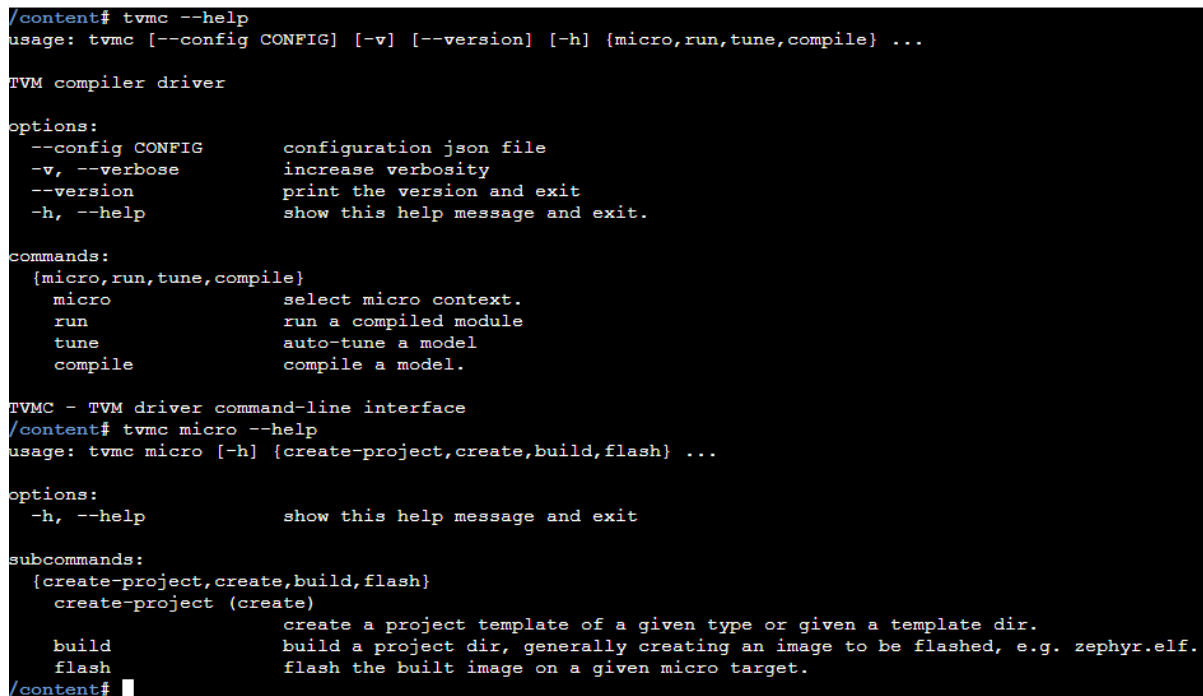
Im ersten Schritt wird Apache TVM mit folgendem Kommando durch Eingabe in der Codezeile installiert:

```
%%shell
# Installs the latest dev build of TVM from PyPI. If you wish to build
# from source, see https://tvm.apache.org/docs/install/from_source.html
pip install apache-tvm -pre
```

Mit dem Terminal kann überprüft werden, ob Apache TVM installiert ist. Das Terminal ist in der kostenlosen Version von Google Colab nicht verfügbar, kann aber mit folgendem Befehl nachinstalliert werden:

```
!pip install colab-xterm  
  
%load_ext colabxterm  
%xterm
```

Dies erzeugt ein Terminal nach der Codezeile. Mit der Eingabe der Befehle *tvmc --help* und *tvmc micro --help* im Terminal wird folgende Ausgabe wie in Abbildung 10 generiert. Diese Abbildung zeigt auch dazu die korrekte Installation von Apache TVM und dem dazugehörigen Add-on microTVM.



```
/content# tvmc --help  
usage: tvmc [--config CONFIG] [-v] [--version] [-h] {micro,run,tune,compile} ...  
  
TVM compiler driver  
  
options:  
  --config CONFIG      configuration json file  
  -v, --verbose         increase verbosity  
  --version             print the version and exit  
  -h, --help           show this help message and exit.  
  
commands:  
  {micro,run,tune,compile}  
    micro              select micro context.  
    run                run a compiled module  
    tune               auto-tune a model  
    compile            compile a model.  
  
TVMC - TVM driver command-line interface  
/content# tvmc micro --help  
usage: tvmc micro [-h] {create-project,create,build,flash} ...  
  
options:  
  -h, --help           show this help message and exit  
  
subcommands:  
  {create-project,create,build,flash}  
    create-project (create)  
                        create a project template of a given type or given a template dir.  
    build             build a project dir, generally creating an image to be flashed, e.g. zephyr.elf.  
    flash             flash the built image on a given micro target.  
/content#
```

Abbildung 10: Google Colab Terminal

5.1.2. Notwendige Python-Pakete

Im nächsten Schritt wird das Paket für die serielle Kommunikation über Python installiert, weil TVM selbst kein Paket für die serielle Kommunikation über Python enthält. Außerdem wird TFLite (TensorFlowLite) benötigt, um Modelle zu laden und Pillow (eine Python Bildgebungsbibliothek), um Beispielbilder vorzubereiten.

```
%%shell  
pip install pyserial==3.5 tflite==2.1 Pillow==9.0 typing_extensions
```

Außerdem werden die folgenden weitere Python-Pakete benötigt:

```

import os
import numpy as np
import pathlib
import json
from PIL import Image
import tarfile

import tvn
from tvn import relay
from tvn.relay.backend import Executor, Runtime
from tvn.contrib.download import download_testdata
from tvn.micro import export_model_library_format
from tvn.relay.op.contrib import cmsisnn
from tvn.micro.testing.utils import create_header_file

```

5.1.3. TFLite Model importieren

In diesem Schritt wird ein **TFLite-Modell** von *Visual Wake Word* aus MLPerf Tiny heruntergeladen und importiert. Dieses Modell verwendet ein 96*96*3 RGB-Bild und bestimmt, ob eine Person auf dem Bild zu sehen ist oder nicht.

A *Visual Wake Word TFLite* (TensorFlow Lite) Model ist ein Modell, das entwickelt wurde, um visuelle Signale in Echtzeit zu verarbeiten und zu klassifizieren, insbesondere im Bereich der Objekterkennung. Ein "Wake Word" ist ein Schlüsselwort oder eine Phrase, die verwendet wird, um ein Gerät zu aktivieren oder eine bestimmte Aktion auszulösen.

Das *TFLite-Modell* basiert auf der TensorFlow Lite Bibliothek von Google, die speziell für mobile und Edge-Geräte optimiert ist. Diese Modelle sind in der Regel kompakt und schnell genug, um auf Smartphones, eingebetteten Geräten oder anderen ressourcenbeschränkten Umgebungen ausgeführt zu werden.

Ein *Visual Wake Word TFLite Modell* wird normalerweise durch Training auf einem Datensatz von Bildern erstellt, die positive und negative Beispiele für das zu erkennendes visuelle Signal enthalten. Das Modell lernt dann, Muster und Merkmale in den Bildern zu erkennen, die mit dem gesuchten Signal korrelieren.

Einmal trainiert, kann das Modell in eine leichtgewichtige Form umgewandelt werden, die für die Verwendung auf mobilen Geräten oder anderen eingebetteten Systemen geeignet ist. Diese Modelle können dann in Echtzeit laufen und visuelle Wake-Words erkennen, wodurch sie vielfältige Anwendungen, wie z.B. die Aktivierung von Geräten durch visuelle Gesten, die Erkennung von bestimmten Objekten oder das Auslösen von Aktionen basierend auf visuellen Signalen vielfältige Anwendungen ermöglichen, wie z.B. die Aktivierung von Geräten durch visuelle.

```

MODEL_URL =
"https://github.com/mlcommons/tiny/raw/bceb91c5ad2e2deb295547d81505721d
3a87d578/benchmark/training/visual_wake_words/trained_models/vww_96_int
8.tflite"
MODEL_NAME = "vww_96_int8.tflite"
MODEL_PATH = download_testdata(MODEL_URL, MODEL_NAME, module="model")

```

```

tflite_model_buf = open(MODEL_PATH, "rb").read()
try:
    import tflite

    tflite_model = tflite.Model.GetRootAsModel(tflite_model_buf, 0)
except AttributeError:
    import tflite.Model

    tflite_model = tflite.Model.Model.GetRootAsModel(tflite_model_buf,
0)

input_shape = (1, 96, 96, 3)
INPUT_NAME = "input_1_int8"
relay_mod, params = relay.frontend.from_tflite(
    tflite_model, shape_dict={INPUT_NAME: input_shape},
dtype_dict={INPUT_NAME: "int8"}
)

```

Der Code lädt ein vortrainiertes TFLite-Modell herunter, liest es ein, analysiert es mit Hilfe der TFLite-Bibliothek und konvertiert es dann in ein Relay-Modul, das für weitere Operationen mit TVM verwendet werden kann, wie beispielsweise Optimierung, Kompilierung oder Ausführung auf verschiedenen Hardware-Zielen.

5.1.4. Generierung des Model Library Format (MLF)

Zunächst wird das Ziel, die Laufzeit und der Executor definiert. Dann wird das Modell für das Zielgerät kompiliert und schließlich der generierte Code und alle erforderlichen Abhängigkeiten in eine einzige Datei exportiert.

```

# We can use TVM native schedules or rely on the CMSIS-NN kernels using
TVM Bring-Your-Own-Code (BYOC) capability.
USE_CMSIS_NN = True

# USMP (Unified Static Memory Planning) performs memory planning of all
tensors holistically to achieve best memory utilization
DISABLE_USMP = False

# Use the C runtime (crt)
RUNTIME = Runtime("crt")

# We define the target by passing the board name to
`tvm.target.target.micro`.
# If your board is not included in the supported models, you can define
the target such as:
# TARGET = tvm.target.Target("c -keys=arm_cpu,cpu -mcpu=cortex-m4")
TARGET = tvm.target.target.micro("stm32l4r5zi")

```

```

# Use the AOT executor rather than graph or vm executors. Use unpacked
API and C calling style.
EXECUTOR = tvm.relay.backend.Executor(
    "aot", {"unpacked-api": True, "interface-api": "c", "workspace-
byte-alignment": 8}
)

# Now, we set the compilation configurations and compile the model for
the target:
config = {"tir.disable_vectorize": True}
if USE_CMSIS_NN:
    config["relay.ext.cmsisnn.options"] = {"mcpu": TARGET.mcpu}
if DISABLE_USMP:
    config["tir.usmp.enable"] = False

with tvm.transform.PassContext(opt_level=3, config=config):
    if USE_CMSIS_NN:
        # When we are using CMSIS-NN, TVM searches for patterns in the
        # relay graph that it can offload to the CMSIS-NN kernels.
        relay_mod = cmsisnn.partition_for_cmsisnn(relay_mod, params,
mcpu=TARGET.mcpu)
    lowered = tvm.relay.build(
        relay_mod, target=TARGET, params=params, runtime=RUNTIME,
executor=EXECUTOR
    )
parameter_size = len(tvm.runtime.save_param_dict(lowered.get_params()))
print(f"Model parameter size: {parameter_size}")

# We need to pick a directory where our file will be saved.
# If running on Google Colab, we'll save everything in
``/root/tutorial`` (aka ``~/tutorial``)
# but you'll probably want to store it elsewhere if running locally.

BUILD_DIR = pathlib.Path("/root/tutorial")

BUILD_DIR.mkdir(exist_ok=True)

# Now, we export the model into a tar file:
TAR_PATH = pathlib.Path(BUILD_DIR) / "model.tar"
export_model_library_format(lowered, TAR_PATH)

```

5.1.5. CMSIS-Bibliothek

CMSIS (*Cortex Microcontroller Software Interface Standard*) ist eine von ARM entwickelte Softwarebibliothek, die eine einheitliche Schnittstelle für die Entwicklung von Embedded-Systemen auf ARM-Cortex-M-Prozessoren bereitstellt. CMSIS erleichtert die Portabilität von Code zwischen verschiedenen Cortex-M-Mikrocontrollern und ermöglicht eine effiziente Programmierung auf niedriger Ebene.

CMSIS 5 ist die neueste Version von CMSIS und bietet eine Vielzahl von Funktionen und Komponenten, die für die Entwicklung von Embedded-Systemen auf Cortex-M-Prozessoren erforderlich sind. Dazu gehören unter anderem einheitliche Header-Dateien, Peripherieinitialisierungs- und Zugriffsfunktionen, Debugging-Unterstützung.

CMSIS-NN (*Neural Network-Kernbibliothek für Cortex-M-Prozessoren*) ist eine spezielle Erweiterung von CMSIS, die Funktionen und Optimierungen für die Ausführung von neuronalen Netzwerken auf Cortex-M-Mikrocontrollern bereitstellt. Diese Bibliothek ermöglicht die Implementierung von KI- und maschinellem Lernen auf eingebetteten Systemen mit begrenzten Ressourcen wie Speicher und Rechenleistung. CMSIS-NN bietet spezielle Funktionen für die Beschleunigung von neuronalen Netzwerken, einschließlich spezieller Funktionen für die Faltung und andere Operationen, die in KI-Modellen verwendet werden.

Insgesamt ermöglichen *CMSIS*, *CMSIS 5* und *CMSIS-NN* die effiziente Entwicklung und Implementierung von Embedded-Systemen und unterstützen insbesondere die Umsetzung von KI- und maschinellem Lernen auf Cortex-M-Prozessoren.

5.1.6. Modellevaluation

Um das erstellte Modell zu testen, werden zwei Beispiele aus der *COCO 2014* Train-Bildern Datenbank verwendet. Diese Bilder werden zunächst in ein rein binäres Format gewandelt und in zwei Header-Dateien gespeichert. **COCO** ist ein umfangreicher Datensatz für die Objekterkennung, Segmentierung und Beschriftung mit den folgenden Merkmalen.

- Segmentierung von Objekten
- Erkennung im Kontext
- Segmentierung von Superpixel-Objekten
- 330K Bilder (>200K beschriftet)
- 1,5 Millionen Objektinstanzen
- 80 Objektkategorien
- 91 Materialkategorien
- 5 Beschriftungen pro Bild
- 250.000 Personen mit Keypoints



Abbildung 11: Bild mit Person [4]



Abbildung 12: Bild ohne Person [4]

```
with tarfile.open(TAR_PATH, mode="a") as tar_file:
    SAMPLES_DIR = "samples"
    SAMPLE_PERSON_URL = (
```

```

        "https://github.com/tlc-pack/web-
data/raw/main/testdata/microTVM/data/vww_sample_person.jpg"
    )
    SAMPLE_NOT_PERSON_URL = "https://github.com/tlc-pack/web-
data/raw/main/testdata/microTVM/data/vww_sample_not_person.jpg"

    SAMPLE_PERSON_PATH = download_testdata(SAMPLE_PERSON_URL,
"person.jpg", module=SAMPLES_DIR)
    img = Image.open(SAMPLE_PERSON_PATH)
    create_header_file("sample_person", np.asarray(img), SAMPLES_DIR,
tar_file)

    SAMPLE_NOT_PERSON_PATH = download_testdata(
        SAMPLE_NOT_PERSON_URL, "not_person.jpg", module=SAMPLES_DIR
    )
    img = Image.open(SAMPLE_NOT_PERSON_PATH)
    create_header_file("sample_not_person", np.asarray(img),
SAMPLES_DIR, tar_file)

```

5.1.7. Modell integrieren

Nach diesem Arbeitsschritt ist alles Nötige da, um das kompilierte Modell in eine IDE zu übernehmen und zu evaluieren. Innerhalb der MLF-Datei(model.tar) findet sich die folgende Dateihierarchie:

```

/root
├── codegen
├── parameters
├── runtime
├── samples
├── src
├── templates
└── metadata.json

```

- Der Ordner codegen enthält den C-Code, den TVM für das Modell generiert hat.
- Der Ordner runtime enthält alle TVM-Abhängigkeiten, die das Ziel benötigt, um den generierten C-Code zu kompilieren.
- Der Ordner samples enthält die beiden generierten Beispieldateien zur Evaluierung des Modells.
- Der Ordner src enthält das Relay-Modul, welches das Modell beschreibt.
- Der Ordner templates enthält zwei Vorlagendateien, die je nach Plattform möglicherweise bearbeitet müssen.
- Die Datei metadata.json enthält Informationen über das Modell, seine Schichten und den Speicherbedarf.

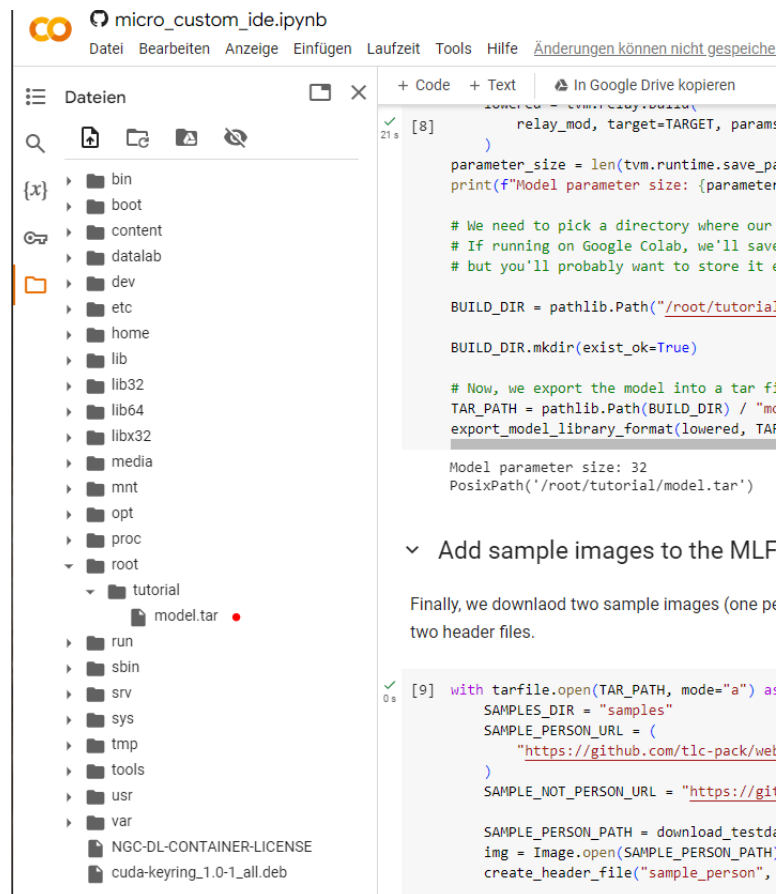


Abbildung 13: Hierarchie der generierten Datei model.tar

5.2 Projekt: microTVM in STM32CubeIDE

Mit **File -> New -> STM32Project** öffnet sich das Auswahlfenster, wie es in Abbildung 14 zu sehen ist. Dort kann unter dem Punkt Board Selector bei Commercial Part Number das benötigte Board eingetragen werden. Im rechten Fenster wird das gesuchte Board ausgewählt.

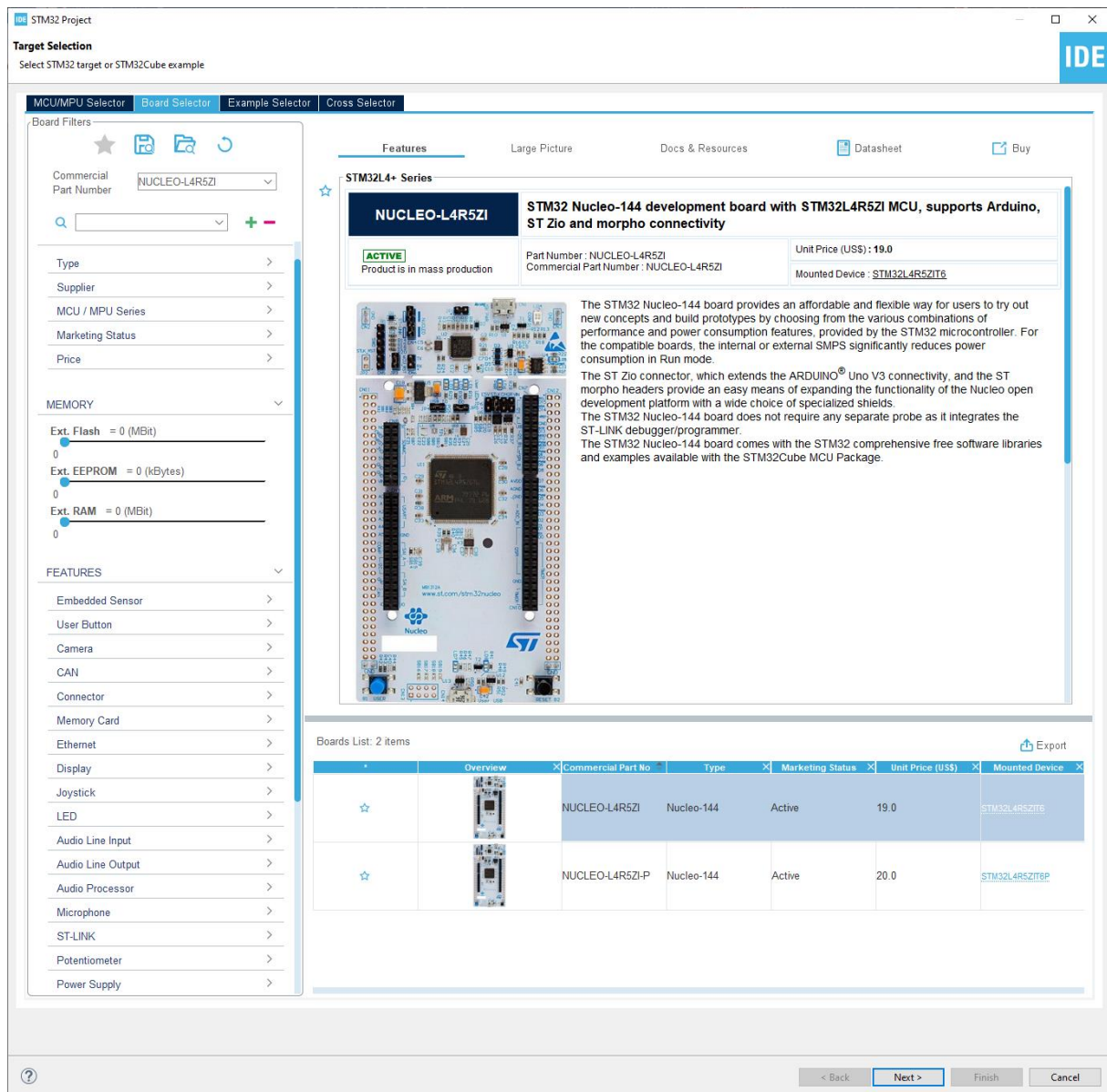


Abbildung 14: STM32 CubeIDE Board Selector

Durch Klicken auf Next kann im nächsten Schritt, wie in Abbildung 15 angezeigt, ein geeigneter Projektname vergeben werden. Die übrigen Einstellungen bleiben so, wie es in den Abbildungen zu sehen ist.

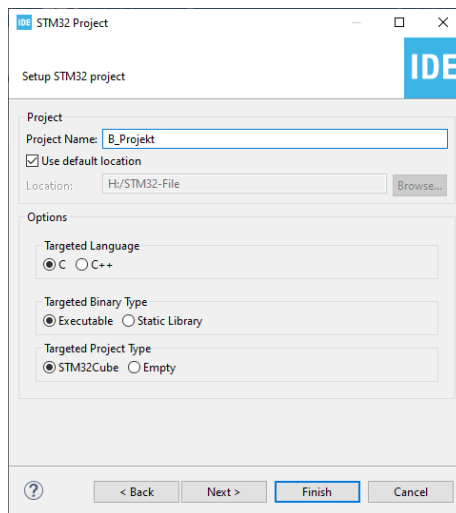


Abbildung 15: Projektname

Durch Finish erscheint die Meldung „Initialize all peripherals with their default“, welche mit Yes quittiert wird. Im Anschluss wird das Projekt für ca. eine Minute generiert. Im nächsten Schritt erscheint die *Pinout & Configuration* des benutzten Boards. Unter dem Punkt **Categories-> LPUART1** wird die Baud Rate auf 115200 Bits/s und Word Length 8 Bits wie in Abbildung 8 geändert. Unter **File -> Save** wird die Änderung gespeichert.

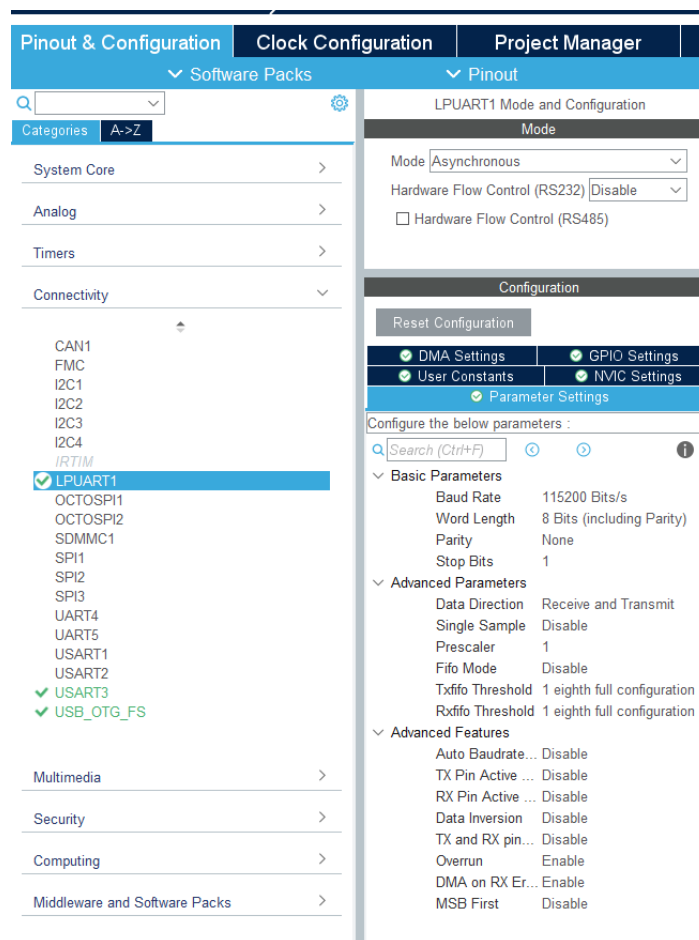


Abbildung 16: UART-Einstellungen

Dies sollte den Code neu generieren und die Datei **main.c** öffnen. Man kann main.c auch im Projekt-Explorer auf der linken Seite unter **projektname -> Core -> Src** finden.

Der folgende Code wird kopiert und in den Abschnitt "Endlosschleife (while (1))" der Hauptfunktion eingefügt, um die Korrektheit zu prüfen.

Hinweis: Es muss sichergestellt werden, dass der Code innerhalb der Abschnitte geschrieben ist, die mit USER CODE BEGIN <...> und USER CODE END <...> gekennzeichnet sind. Der Code außerhalb dieser Abschnitte wird gelöscht, wenn der Initialisierungscode neu generiert wird.

```
HAL_GPIO_TogglePin(LD2_GPIO_Port, LD2_Pin);  
HAL_UART_Transmit(&hlpuart1, "Hello World.\r\n", 14, 100);  
HAL_Delay(1000);
```

Dieser Codeausschnitt führt einige grundlegende Operationen auf einem Mikrocontroller aus, wie das Umschalten eines GPIO-Pins, das Senden einer Zeichenkette über UART und das Einfügen einer Verzögerung. Es könnte Teil eines größeren Programms sein, das eine bestimmte Funktionalität auf einem Mikrocontroller implementiert.

Mit Rechtsklick auf den Projektnamen wird mit Build Project das Projekt generiert. Oben in der Auswahlleiste kann durch Run die Ausführung des Programms gestartet werden. Dieser Test führt dazu, dass die LD2-LED blau blinkt. Und bei jedem Aufleuchten gibt die Konsole, wie in der Abbildung 10 dargestellt, eine „Hello World“ Ausgabe raus.

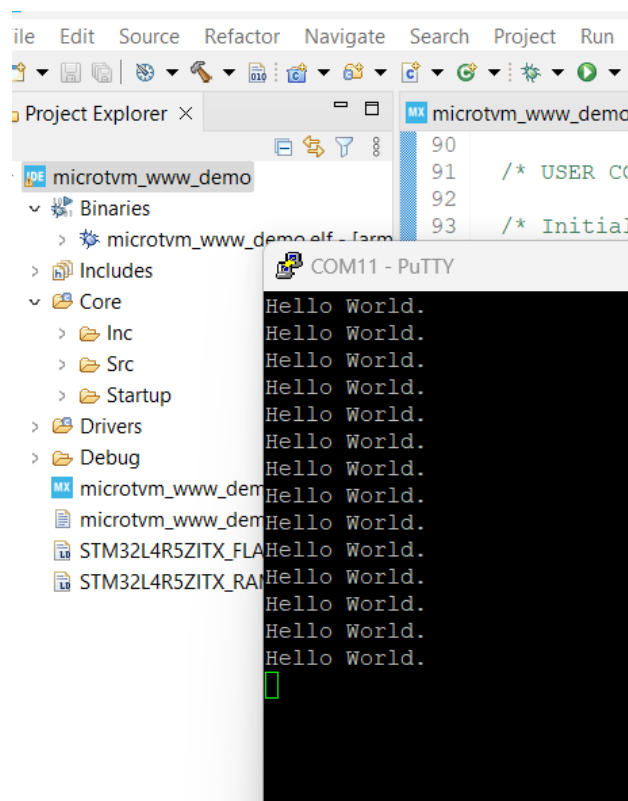


Abbildung 17: Hello World. Ausgabe im Terminalprogramm PuTTY

Um die Ausgabe der Konsole des Boards zu sehen, kann unter Open console -> command shelf console ein Zugang erstellt werden. Abbildung 18 zeigt, welches Fenster sich öffnet. Als Connection Type wird Serial Port ausgewählt. Für Connection name wird mit New ein neues Fenster geöffnet, wie in Abbildung 19 zu sehen ist.

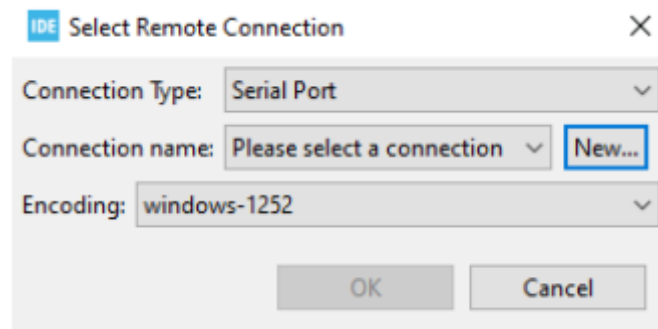


Abbildung 18: Remote Connection

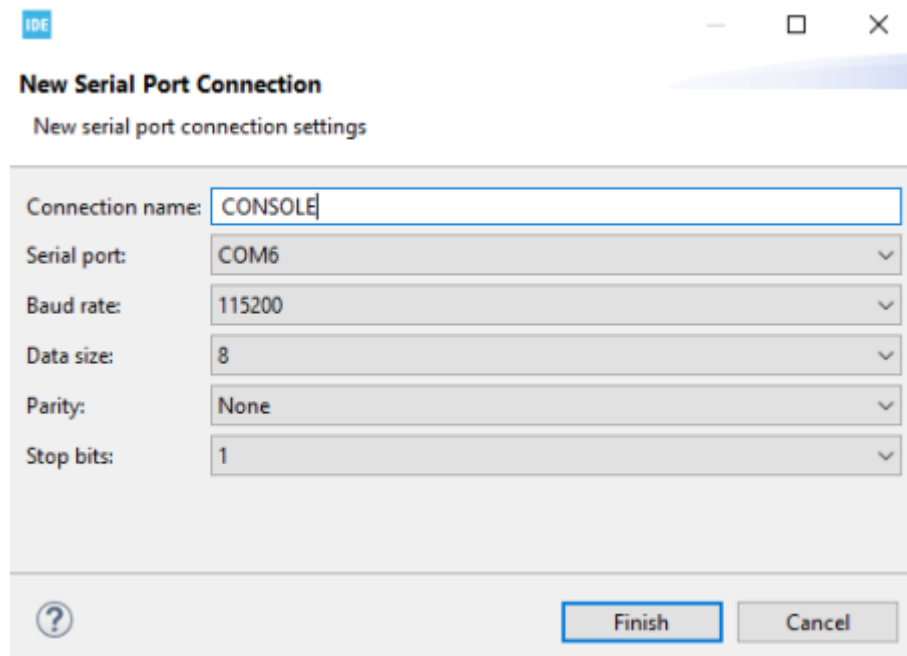


Abbildung 19: Serial Port Connection

Hier kann ein beliebiger Name für die Verbindung eingegeben werden. Die anderen Standardeinstellungen bleiben unverändert und mit **Finish->OK** wird die Konsolenausgabe erzeugt. Um das kompilierte Modell in das Projekt zu integrieren, müssen die folgenden Schritte durchgeführt werden. Mit Rechtsklick auf den Projektnamen wird **Properties -> C / C++ General -> Paths and Symbols -> Source Location -> Link Folder..** geöffnet. Danach öffnet sich das folgende Fenster wie in Abbildung 20 dargestellt.

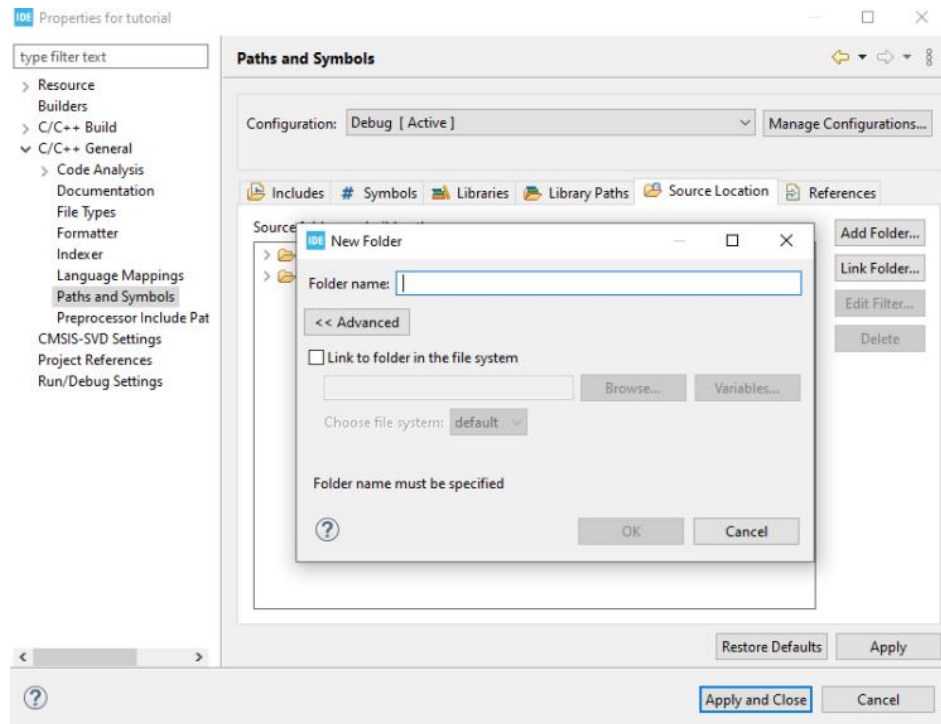


Abbildung 20: Paths and Symbols

Im nächsten Schritt wird ein Haken bei „Link to folder in the file system“ gesetzt. Mit Browse.. wird der Speicherort des heruntergeladenen Modells von Google Colab ausgewählt und mit OK bestätigt. In diesem Projekt wird die CMSIS-NN Bibliothek verwendet, die über den Link „<https://github.com/ARM-software/CMSIS-NN>“ heruntergeladen werden kann und deren Verzeichnis wie oben beschrieben ausgewählt wird.

Nach den beiden Verlinkungen wird der Vorgang mit Apply & Close abgeschlossen. Im Projekt Explorer soll nach Durchführung der beschriebenen Konfigurationsschritte die folgende Struktur wie in Abbildung 21 aufgelistet sein.

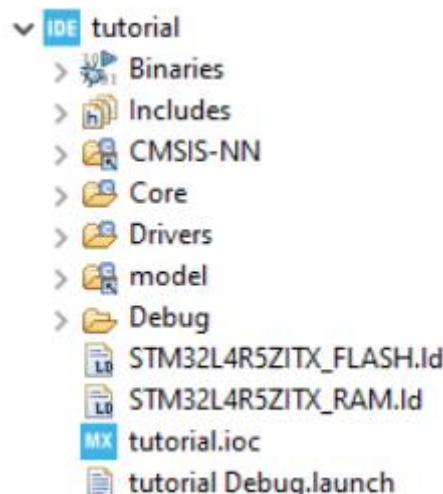


Abbildung 21: Projektaufstellung

Im nächsten Schritt wird mit Rechtsklick auf den **Projektnamen -> properties -> C /C++ Build -> Settings - > MCU GCC Compiler -> Include paths** das folgende Fenster (siehe Abbildung 22) geöffnet.

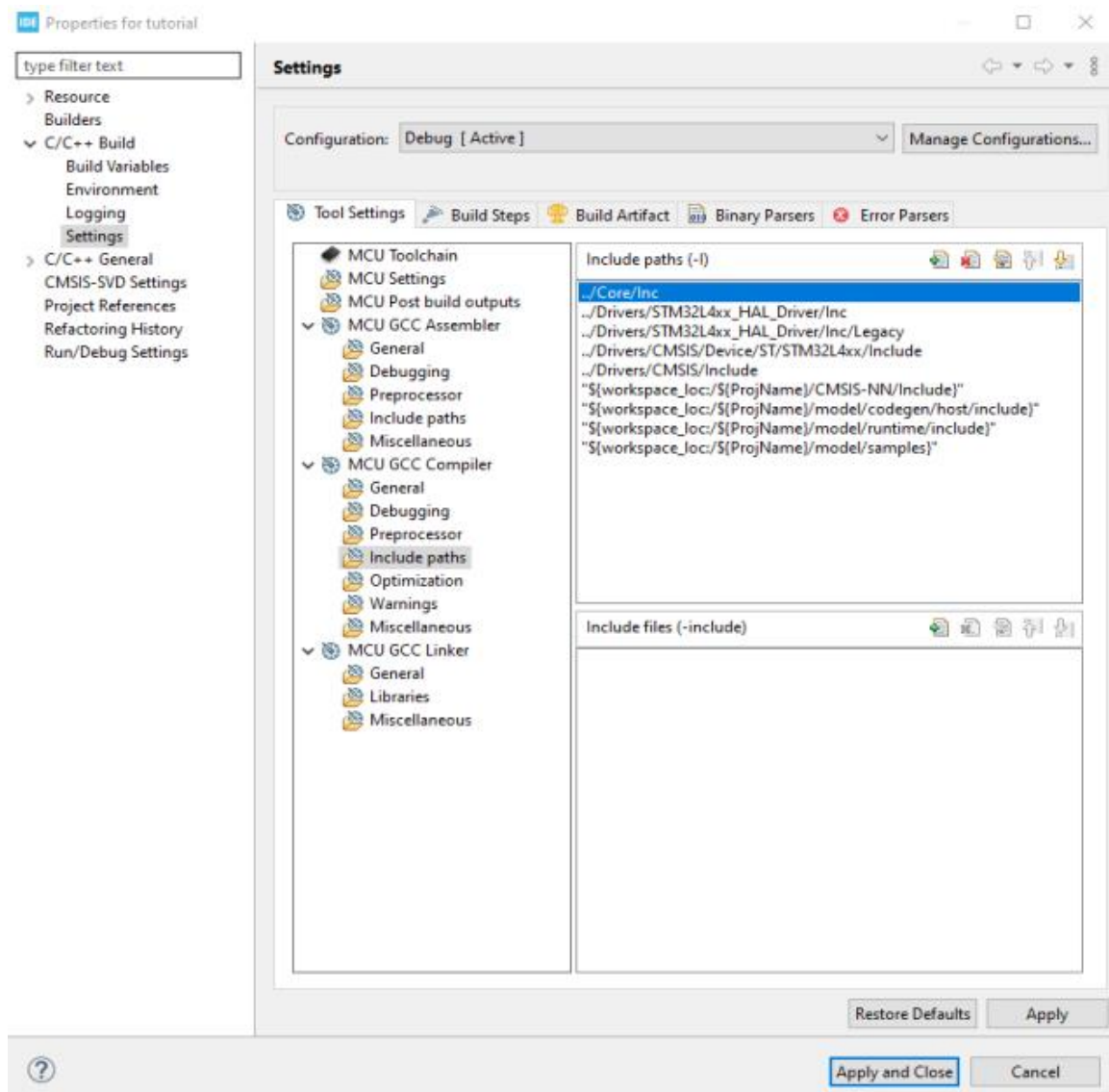


Abbildung 22: Settings - include Paths

Mit dem Klick auf „+“ -> Workspace werden folgende Ordner ausgewählt:

- model/runtime/include
- model/codegen/host/include
- model/samples
- CMSIS-NN/Include

Mit *Apply & Close* wird der Schritt abgeschlossen.

Im weiteren Schritt wird die Datei `crt_config.h.template` von `model/templates` in den Ordner `Core/Inc` Datei verschoben und in `crt_config.h` umbenannt. Zusätzlich wird auch die `platform.c.template` Datei von `model/templates` in den Ordner `Core/Src` Datei verschoben und der Name auf `platform.c` geändert.

In die `platform.c` Datei wird die Compiler-Direktive **`#define TVM_WORKSPACE_SIZE_BYTES 1024`**, um den benötigten Speicher einzustellen.

Der Unterordner Tests von CMSIS-NN wird mit **Rechtsklick -> Resource Configurations -> Exclude from Build** aus dem Compilervorgang ausgeschlossen. Anschließend wird jeweils ein Haken bei Debug und Release gesetzt und mit OK bestätigt.

Nach Durchführung der Schritte sollte die Ordnerstruktur der Darstellung in Abbildung 23 entsprechen.

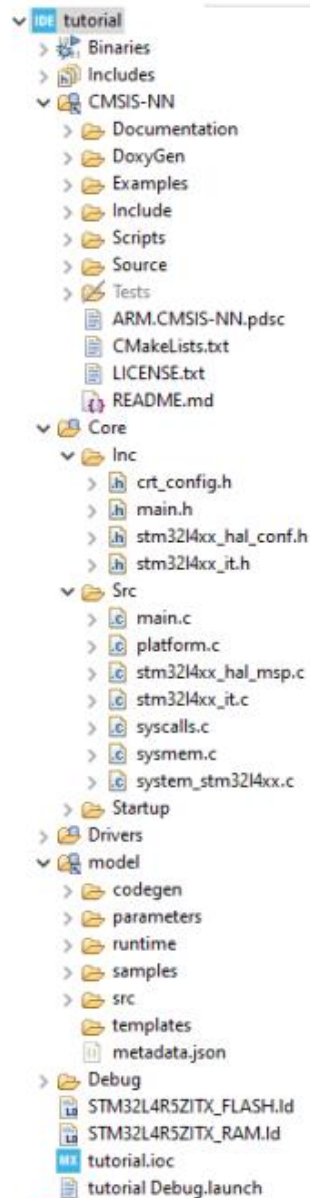


Abbildung 23: Ordnerstruktur

Da der Ordner Drivers/CMSIS/Include veraltet ist, wird vom Web- Link https://github.com/ARM-software/CMSIS_5 die neuere Version heruntergeladen. Die Datei wird entpackt und der ältere Include-Ordner wird durch den neueren Include-Ordner unter dem Pfad CMSIS/Core/Include im Projekt ersetzt. Nun werden Änderungen in dem *main.c*-Ordner getroffen. Die folgenden Header-Dateien werden in das Projekt integriert.

```

/* Private includes -----*/
/* USER CODE BEGIN Includes */

#include <stdio.h>

#include <string.h>

#include <stdarg.h>

#include "tvmgen_default.h"

#include "sample_person.h"

#include "sample_not_person.h"

/* USER CODE END Includes */

```

Danach wird der folgende Code, wie in Abbildung 19 gezeigt, in die Hauptfunktion vor die Endlosschleife kopiert. Aber bevor werden die entsprechende Variable als global (vor die Hauptfunktion) definiert:

```

uint8_t receivedData[10];
uint8_t bufferData[96*96*3];
uint16_t Index;

/**
 * @brief The application entry point.
 * @retval int
 */
int main(void)
{

/* MCU Configuration-----*/

/* Reset of all peripherals, Initializes the Flash interface and the Systick. */
HAL_Init();

/* USER CODE BEGIN Init */

/* USER CODE END Init */

```

```

/* Configure the system clock */
SystemClock_Config();

/* USER CODE BEGIN SysInit */

/* USER CODE END SysInit */

/* Initialize all configured peripherals */
MX_GPIO_Init();
MX_LPUART1_UART_Init();
MX_USART3_UART_Init();
MX_USB_OTG_FS_PCD_Init();
/* USER CODE BEGIN 2 */

TVMPlatformInitialize();

signed char output[2];

struct tvmgcn_default_inputs inputs = {
    .input_1_int8 = (void*)&sample_person,
};

struct tvmgcn_default_outputs outputs = {
    .Identity_int8 = (void*)&output,
};

char msg[] = "Evaluating VWV model using microTVM:\r\n";
HAL_UART_Transmit(&hlpuart1, msg, strlen(msg), 100);

uint8_t sample = 0;
uint32_t timer_val;

char buf[50];
uint16_t buf_len;

```

Der folgende Code gibt in der Konsole einen Text aus, der darüber Auskunft gibt, ob eine Person in dem empfangenen Bilderarray (**bufferData**) erkannt wurde. Die Übermittlung und Speicherung der Bilddaten in das Array werden später erläutert. (void*)bufferData übergibt einen Pointer auf den Beginn des Arrays. Diese wird in die Endlosschleife kopiert:

```

if (HAL_UART_Receive(&hlpuart1, bufferData, sizeof(bufferData), -1) == HAL_OK) {
    inputs.input_1_int8 = (void*)bufferData;
    timer_val = HAL_GetTick();

    tvmgem_default_run(&inputs, &outputs);

    timer_val = HAL_GetTick() - timer_val;

    if (output[0] > output[1])
    {
        buf_len = sprintf(buf, "Person not detected, inference time = %lu
ms\r\n", timer_val);
    }
    else
    {
        buf_len = sprintf(buf, "Person detected, inference time = %lu ms\r\n",
timer_val);
    }

    HAL_UART_Transmit(&hlpuart1, buf, buf_len, 100);
}

Index++;

if (HAL_UART_Receive(&hlpuart1, bufferData, sizeof(bufferData), -1) == HAL_OK) {
    inputs.input_1_int8 = (void*)bufferData;
    timer_val = HAL_GetTick();

    tvmgem_default_run(&inputs, &outputs);

    timer_val = HAL_GetTick() - timer_val;

    if (output[0] > output[1])
    {
        buf_len = sprintf(buf, "Person not detected, inference time = %lu ms\r\n", timer_val);
    }
    else
    {
        buf_len = sprintf(buf, "Person detected, inference time = %lu ms\r\n", timer_val);
    }

    HAL_UART_Transmit(&hlpuart1, buf, buf_len, 100);
}

Index++;

```

Abbildung 24: Code in der Endlosschleife

Um TVM-Laufzeitfehler in der Konsole sehen zu können, wird der folgende Code in die Datei kopiert.

```

void TVMLogf(const char* msg, ...) {
    char buffer[128];

```

```

int size;
va_list args;
va_start(args, msg);
size = TVMPlatformFormatMessage(buffer, 128, msg, args);
va_end(args);
HAL_UART_Transmit(&hlpuart1, buffer, size, 100);
}

```

Als abschließende Einstellung wird mit Rechtsklick auf den **Projektnamen** -> **C/C++ Build** -> **Settings** -> **MCU GCC Compiler** -> **Optimization**, der Optimization level auf "Optimize more (-O2)" gesetzt, um eine bessere Laufzeit zu erhalten.

5.3 Erstellung einer GUI zur Auswertung von Bildern mit Qt-Creator

Zuerst wird mit **Datei** → **Neues Projekt** ein neues Projekt erstellt. Dann öffnet sich ein Fenster (Abbildung 25), in dem die Anwendung „Qt Widgets-Anwendung“ ausgewählt wird.

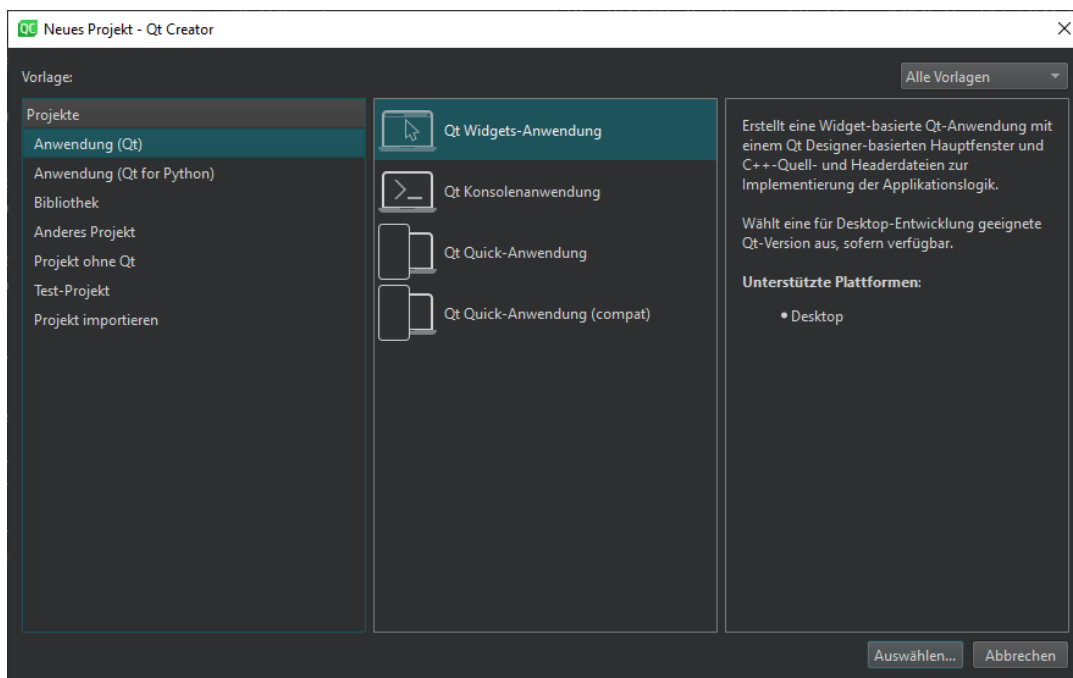


Abbildung 25: Neues Projekt auf Qt Creator

Danach wird im Pfad ein geeigneter Projektname vergeben.

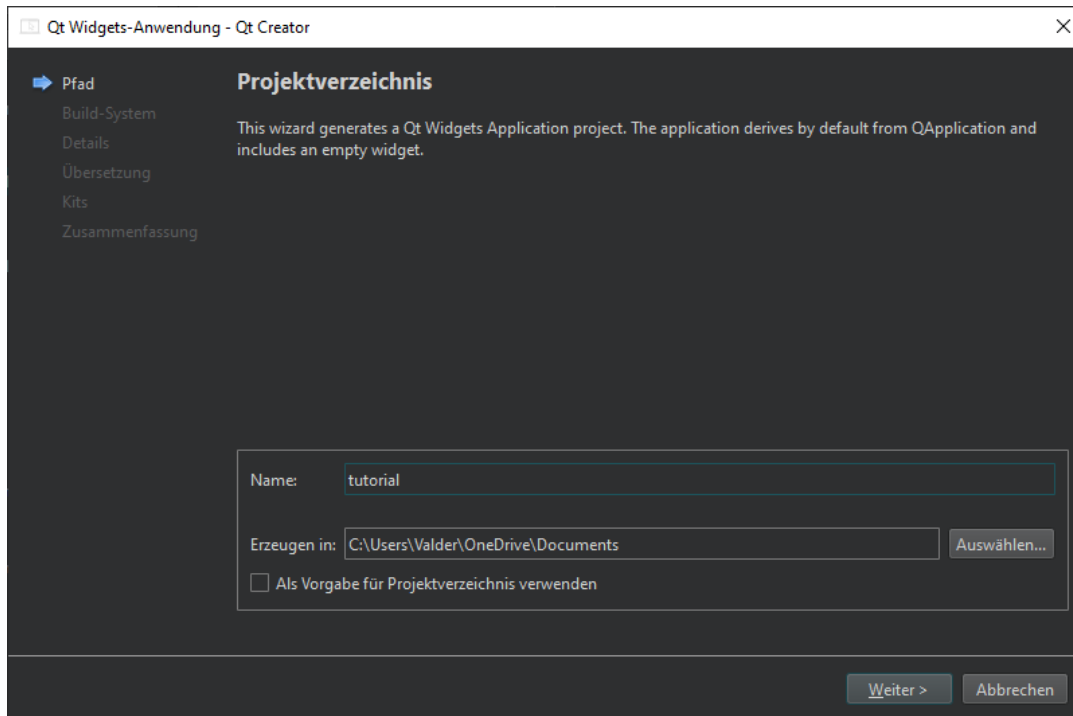


Abbildung 26: Projektname

Durch Klicken auf *Weiter* erscheint der Build-System Bereich, Dort kann man auswählen, ob das Projekt mit **qmake** oder **CMake** (Abbildung 27) erstellt werden soll. In diesem Projekt wird **qmake** verwendet.

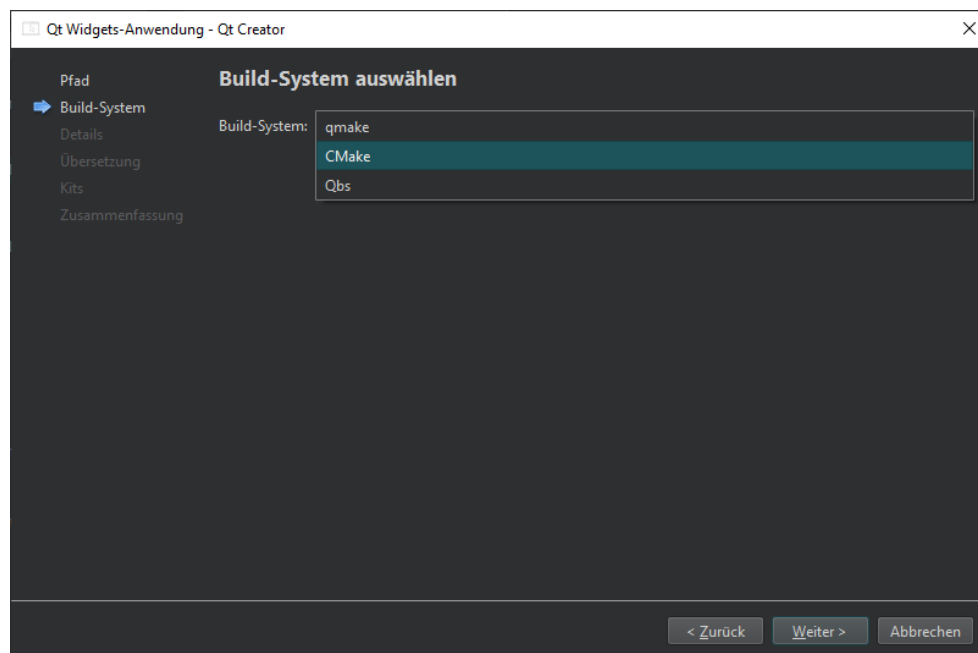


Abbildung 27: Build-System

Im nächsten Schritt erscheint der **Kits-Bereich**, dort wird der passende Kit für das Projekt ausgewählt. Ein Kit besteht aus einer Reihe von Werten, die eine Umgebung definieren, z. B. ein Gerät, eine Werkzeugkette, eine Qt-Version und ein Debugger-Befehl, welche verwendet werden soll.

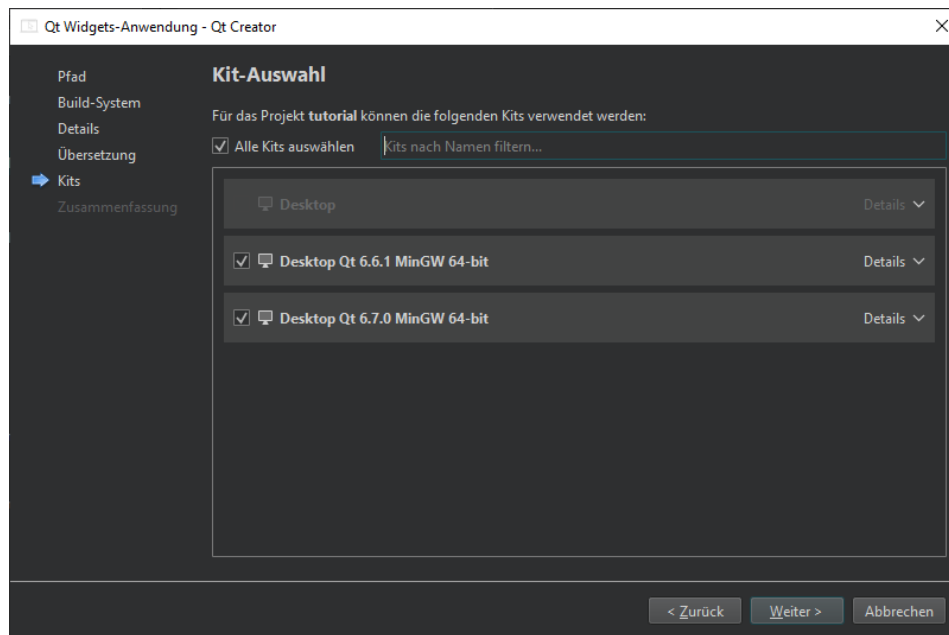


Abbildung 28: Kits-Auswahl

Am Ende der Projekterstellung erscheint eine Zusammenfassung, an Hand derer man sehen kann, welche Dateien hinzugefügt werden.

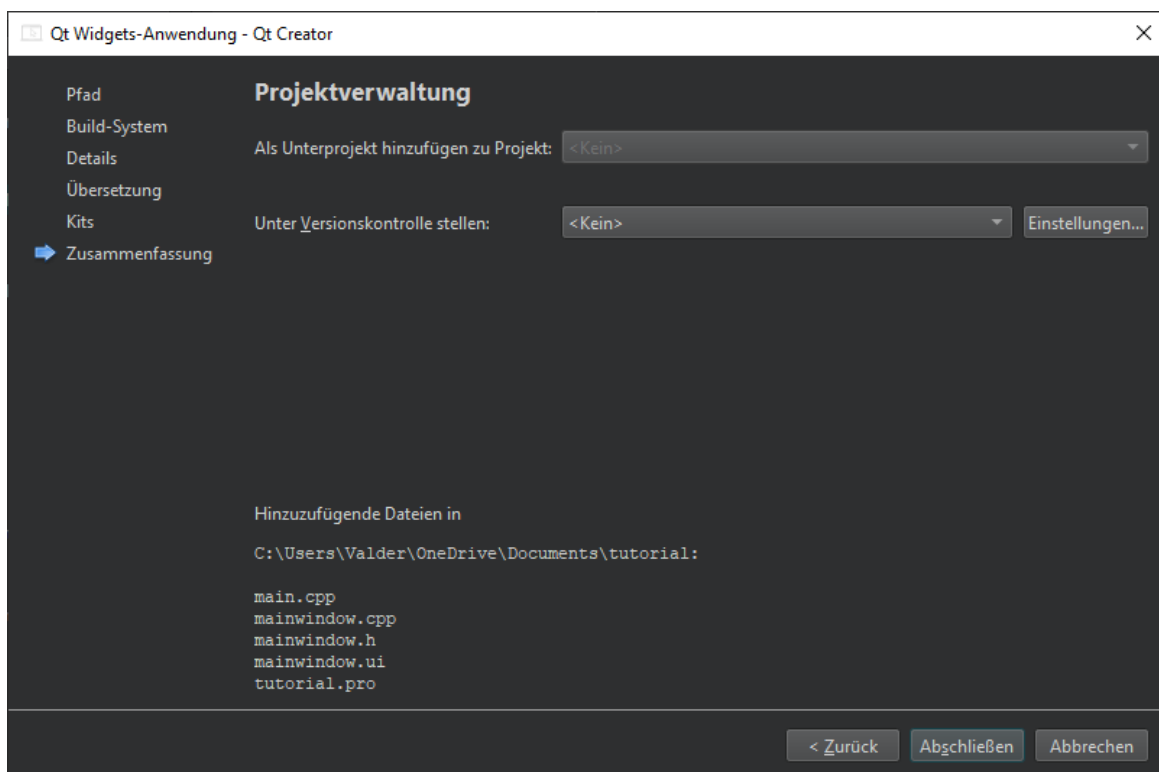


Abbildung 29: Zusammenfassung der Projekt-Erstellung

In diesem Schritt werden zusätzliche Qt-Pakete geladen, die für die Durchführung des Projekts notwendig sind. Dazu wird im Menü **Extras → QT Maintenance Tool → Maintenance Tool Starten** ein Haken bei **Qt Serial Port** im Bereich Auswahl Komponenten --> **Qt → Qt 6.6.1 oder neue Qt Version → Additional Tool → Qt Serial Port** gesetzt (Abbildung 30).

Qt Serial Port bietet grundlegende Funktionen für die Konfiguration, E/A-Operationen und das Abrufen und Setzen der Steuersignale einer RS-232 Schnittstelle.

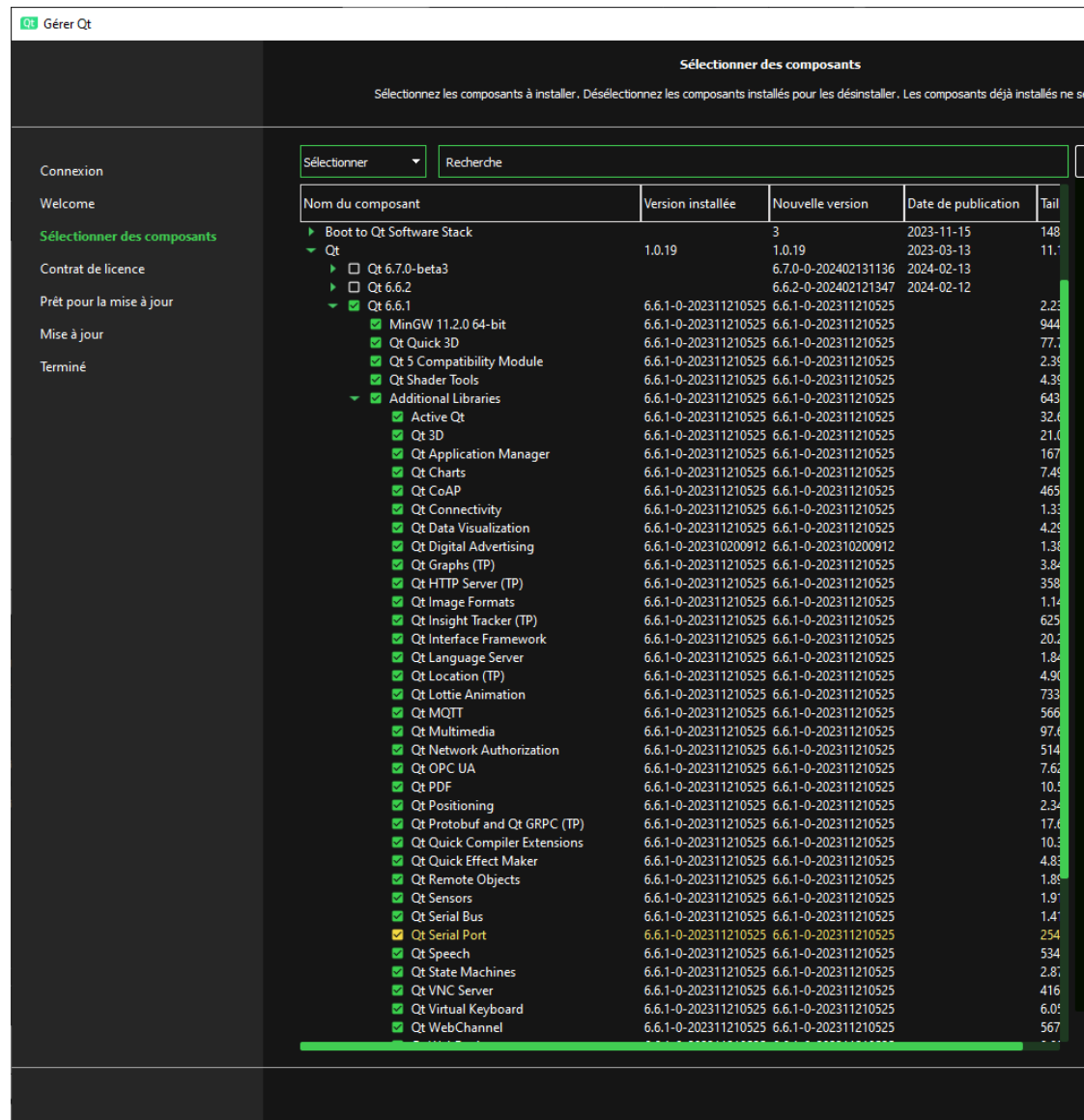


Abbildung 30: Qt-Verwaltung (Maintenance Tool)

Im Projekt wird als nächstes die Datei *main.cpp* geöffnet, diese enthält üblicherweise die Deklaration der Hauptklasse, Funktionen, Signale und Slots sowie den Befehl zur Anzeige des Hauptfensters auf dem Bildschirm, während *mainwindow.cpp* die Implementierung dieser Funktionen und Signale und Slots enthält.

```
1 #include "mainwindow.h"
2
3 #include <QApplication>
4
5 int main(int argc, char *argv[])
6 {
7     QApplication a(argc, argv);
8     MainWindow w;
9     w.show();
10    return a.exec();
11 }
12
```

Abbildung 31: main.cpp

Im Unterverzeichnis Formulardateien wird die *mainwindow.ui* geöffnet und ,wie in Abbildung 32 dargestellt, mit Push Buttons ,Combobox, Label Line Edit, Text Browser und weiteren Fensterbestandteilen ausgefüllt. Die Datei "*mainwindow.ui*" enthält die Benutzeroberflächendefinition für die Anwendung, die mit dem Qt-Framework erstellt wird. Diese Datei wird normalerweise vom Qt Designer generiert bei dem es sich um ein grafisches Werkzeug zur Erstellung von Benutzeroberflächen handelt.

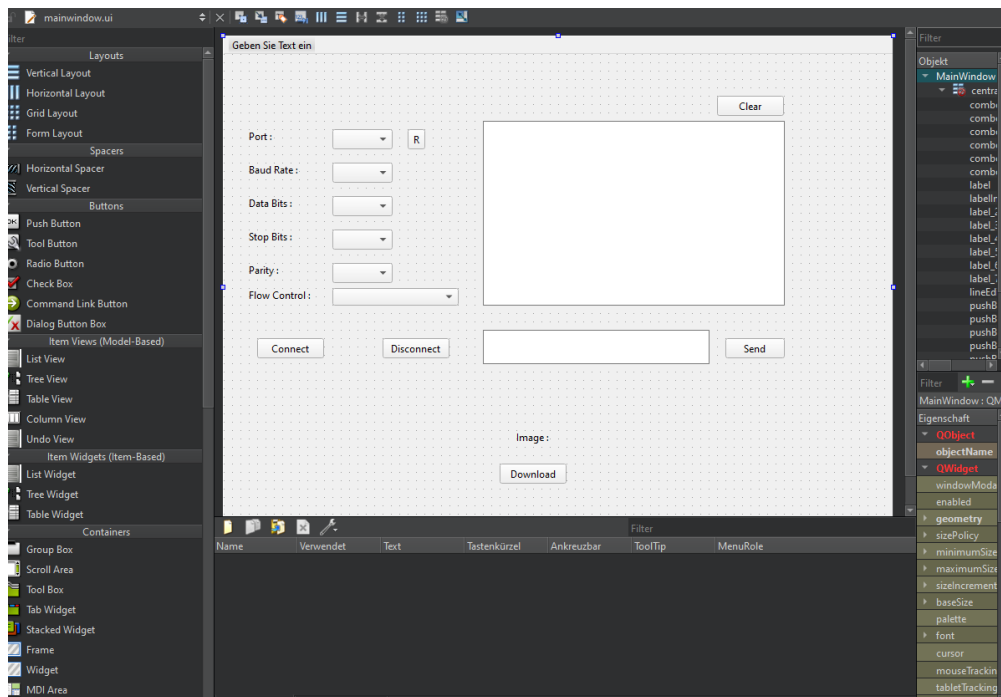


Abbildung 32: Editierbare Darstellung der Datei mainwindow.ui

Im nächsten Schritt wird die „.pro“-Datei bearbeitet und die serielle Bibliothek für die Schnittstelle wie folgt eingebunden: „`QT += core serialport`“. Die .pro-Datei wird von *qmake* verwendet, um Makefiles oder Projektdateien für die verschiedenen Build-Tools zu generieren (z.B. Makefile für Unix, Visual Studio-Projektdatei für Windows).

```

1  QT      += core gui serialport
2
3  greaterThan(QT_MAJOR_VERSION, 4): QT += widgets
4
5  CONFIG += c++17
6
7  # You can make your code fail to compile if it uses deprecated APIs.
8  # In order to do so, uncomment the following line.
9  #DEFINES += QT_DISABLE_DEPRECATED_BEFORE=0x060000    # disables all the APIs deprecated before Qt 6.0
10
11 SOURCES += \
12 |     main.cpp \
13 |     mainwindow.cpp
14
15 HEADERS += \
16 |     mainwindow.h
17
18 FORMS += \
19 |     mainwindow.ui
20
21 # Default rules for deployment.
22 qnx: target.path = /tmp/${TARGET}/bin
23 else: unix:!android: target.path = /opt/${TARGET}/bin
24 !isEmpty(target.path): INSTALLS += target
25

```

Abbildung 33: Inhalt der pro-Datei

Die **mainwindow.h**-Datei ist eine Headerdatei, welche die Deklarationen für die Klasse **MainWindow** enthält. Diese Klasse repräsentiert in der Regel das Hauptfenster Ihrer Qt-Anwendung. Diese Headerdatei dient dazu, die Schnittstelle und die Funktionalität des Hauptfensters der Qt-Anwendung zu definieren. Die Implementierung der Methoden und Slots erfolgt in der entsprechenden **mainwindow.cpp**-Datei.

The image shows a code editor window titled 'mainwindow.h'. The code is written in C++ and defines the 'MainWindow' class. It includes Qt headers for QMainWindow, QSerialPort, and QSerialPortInfo. The class is in the 'Ui' namespace and inherits from QMainWindow. It declares several private slots for button clicks and a method for displaying an image. The public section contains the constructor and destructor. The private section contains member variables for a UI pointer, serial port objects, and buffers. The code is enclosed in a preprocessor guard.

```

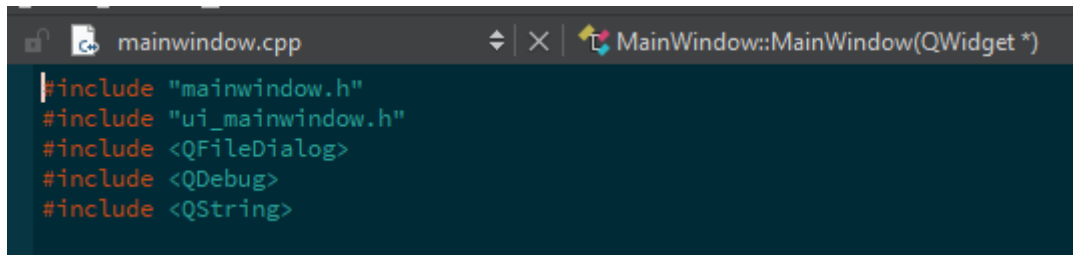
1  #ifndef MAINWINDOW_H
2  #define MAINWINDOW_H
3
4  #include <QMainWindow>
5  #include <QSerialPort>
6  #include <QSerialPortInfo>
7
8  QT_BEGIN_NAMESPACE
9  namespace Ui {
10     class MainWindow;
11 }
12 QT_END_NAMESPACE
13
14 class MainWindow : public QMainWindow
15 {
16     Q_OBJECT
17
18     private slots:
19         void on_pushButton_2_clicked();
20         void receiveMessage();
21         void on_pushButton_clicked();
22         void on_pushButton_6_clicked();
23         void on_pushButton_3_clicked();
24         void on_pushButton_4_clicked();
25         void on_pushButton_5_clicked();
26         //void displayImage(const QString &filePath);
27         void displayImage(const QImage &img);
28
29     public:
30         MainWindow(QWidget *parent = nullptr);
31         ~MainWindow();
32
33     private:
34         Ui::MainWindow *ui;
35         QSerialPort serialPort;
36         QSerialPortInfo info;
37         QString buffer;
38         QString code;
39         int codeSize;
40 };
41 #endif // MAINWINDOW_H
42

```

Abbildung 34: mainwindow.h

In der Quelldatei „mainwindow.cpp“ wird:

- Als erstes notwendige „Include-Direktiven“ hinzugefügt. Diese fügen die Definitionen von Klassen und Funktionen aus den entsprechenden Qt-Bibliotheken in die mainwindow.cpp-Datei ein, damit wir sie verwenden können.



```
mainwindow.cpp
MainWindow::MainWindow(QWidget *)

#include "mainwindow.h"
#include "ui_mainwindow.h"
#include <QFileDialog>
#include <QDebug>
#include <QString>
```

Abbildung 35: mainwindow.cpp - Header File und Pakete laden

- In dem Default-Konstruktor wird die Funktionalität der seriellen Schnittstelle konkretisiert und z.B. Port , Baudrate, Data Bits, Start/Stop Bits und Flow Control mit Comboboxen definiert. (Abbildung 37)



```
mainwindow.cpp
MainWindow::MainWindow(QWidget *)
CRLF

MainWindow::MainWindow(QWidget *parent)
: QMainWindow(parent)
, ui(new Ui::MainWindow)
{
    ui->setupUi(this);
    // Disable maximizing
    setFixedSize(width(), height());

    // Adding title for widget
    QWidget::setWindowTitle("QTerminal");

    // Ports
    QList<QSerialPortInfo> ports = info.availablePorts();
    QList<QString> stringPorts;
    for(int i = 0 ; i < ports.size() ; i++){
        stringPorts.append(ports.at(i).portName());
    }
    ui->comboBox->addItem(stringPorts);

    // Baud Rate Ratios
    QList<qint32> baudRates = info.standardBaudRates(); // What baudrates does my computer support ?
    QList<QString> stringBaudRates;
    for(int i = 0 ; i < baudRates.size() ; i++){
        stringBaudRates.append(QString::number(baudRates.at(i)));
    }
    ui->comboBox_2->addItem(stringBaudRates);

    // Data Bits
    ui->comboBox_3->addItem("5");
    ui->comboBox_3->addItem("6");
    ui->comboBox_3->addItem("7");
    ui->comboBox_3->addItem("8");

    // Stop Bits
    ui->comboBox_4->addItem("1 Bit");
    ui->comboBox_4->addItem("1,5 Bits");
    ui->comboBox_4->addItem("2 Bits");

    // Parities
    ui->comboBox_5->addItem("No Parity");
    ui->comboBox_5->addItem("Even Parity");
    ui->comboBox_5->addItem("Odd Parity");
    ui->comboBox_5->addItem("Mark Parity");
    ui->comboBox_5->addItem("Space Parity");

    //Flow Controls
    ui->comboBox_6->addItem("No Flow Control");
    ui->comboBox_6->addItem("Hardware Flow Control");
    ui->comboBox_6->addItem("Software Flow Control");

    //connect(ui->pushButton_3, SIGNAL(clicked()), this, SLOT(on_pushButton_3_clicked()));
}
```

Abbildung 36: mainwindow.cpp- Default Konstruktor der Klasse MainWindow

- Im Destruktor wird der Befehl *delete ui*; verwendet.

```
MainWindow::~MainWindow()
{
    delete ui;
}
```

Abbildung 37: mainwindow.cpp - Destruktor

Wenn der Button „**Connect**“ angeklickt wird, wird der in Abbildung 38 und 39 dargestellte Code ausgeführt.

Diese Methode behandelt die Logik für die Verbindungsherstellung über den seriellen Port basierend auf den vom Benutzer ausgewählten Einstellungen in den ComboBoxen der GUI.

QString portName = ui->comboBox->currentText (); Hier wird der aktuelle Text des ausgewählten Elements (Portname) in einem ComboBox-Widget (**ui->comboBox**) abgerufen und in der Variablen **portName** gespeichert.

serialPort.setPortName(portName); Der Portname für das SerialPort-Objekt (**serialPort**) wird auf den in portName gespeicherten Wert gesetzt.

serialPort.open(QIODevice::ReadWrite); Der serielle Port wird geöffnet, um Lese- und Schreibzugriffe zu ermöglichen.

Es wird überprüft, ob der Port erfolgreich geöffnet wurde. Wenn nicht, wird der Text in einem Textbrowser-Widget (**ui->textBrowser**) auf **Rot** gesetzt und eine Fehlermeldung ("**!!! Something went Wrong !!!**") angehängt.

Wenn der Port erfolgreich geöffnet wurde, werden die Baudrate, die Datenbits, die Stopbits, die Parität und die Flusssteuerung basierend auf den ausgewählten Optionen in den entsprechenden ComboBox-Widgets (**ui->comboBox_2**, **ui->comboBox_3**, **ui->comboBox_4**, **ui->comboBox_5**, **ui->comboBox_6**) konfiguriert.

Eine Verbindung wird zwischen dem Signal **readyRead()** des SerialPort-Objekts und dem Slot **receiveMessage()** hergestellt. Wenn Daten auf dem seriellen Port verfügbar sind, wird die Methode **receiveMessage()** aufgerufen.

Eine Erfolgsmeldung ("**CONNECTED !!**") wird im Textbrowser-Widget angezeigt, wenn die Verbindung erfolgreich hergestellt wurde.

```

void MainWindow::on_pushButton_clicked()
{
    QString portName = ui->comboBox->currentText();
    serialPort.setPortName(portName);

    serialPort.open(QIODevice::ReadWrite);

    if(!serialPort.isOpen()){
        ui->textBrowser->setTextColor(Qt::red);
        ui->textBrowser->append("!!!! Something went Wrong !!!!");
    }
    else {

        QString stringbaudRate = ui->comboBox_2->currentText();
        int intbaudRate = stringbaudRate.toInt();
        serialPort.setBaudRate(intbaudRate);

        QString dataBits = ui->comboBox_3->currentText();
        if(dataBits == "5 Bits") {
            serialPort.setDataBits(QSerialPort::Data5);
        }
        else if((dataBits == "6 Bits")) {
            serialPort.setDataBits(QSerialPort::Data6);
        }
        else if(dataBits == "7 Bits") {
            serialPort.setDataBits(QSerialPort::Data7);
        }
        else if(dataBits == "8 Bits"){
            serialPort.setDataBits(QSerialPort::Data8);
        }

        QString stopBits = ui->comboBox_4->currentText();
        if(stopBits == "1 Bit") {
            serialPort.setStopBits(QSerialPort::OneStop);
        }
        else if(stopBits == "1,5 Bits") {
            serialPort.setStopBits(QSerialPort::OneAndHalfStop);
        }
        else if(stopBits == "2 Bits") {
            serialPort.setStopBits(QSerialPort::TwoStop);
        }
    }
}

```

Abbildung 38: Methode 'on_pushButton_clicked' der Klasse MainWindow

```

QString parity = ui->comboBox_5->currentText();
if(parity == "No Parity"){
    serialPort.setParity(QSerialPort::NoParity);
}
else if(parity == "Even Parity"){
    serialPort.setParity(QSerialPort::EvenParity);
}
else if(parity == "Odd Parity"){
    serialPort.setParity(QSerialPort::OddParity);
}
else if(parity == "Mark Parity"){
    serialPort.setParity(QSerialPort::MarkParity);
}
else if(parity == "Space Parity") {
    serialPort.setParity(QSerialPort::SpaceParity);
}

QString flowControl = ui->comboBox_6->currentText();
if(flowControl == "No Flow Control") {
    serialPort.setFlowControl(QSerialPort::NoFlowControl);
}
else if(flowControl == "Hardware Flow Control") {
    serialPort.setFlowControl(QSerialPort::HardwareControl);
}
else if(flowControl == "Software Flow Control") {
    serialPort.setFlowControl(QSerialPort::SoftwareControl);
}

//ui->radioButton->isChecked()=true;

connect(&serialPort,SIGNAL(readyRead()),this,SLOT(receiveMessage()));
QString message = "CONNECTED !!";
ui->textBrowser->setTextColor(Qt::darkCyan); // Color of message to send is green.
ui->textBrowser->append(message);
}

}

```

Abbildung 39: Methode 'on_pushButton_clicked' der Klasse MainWindow

- **receiveMessage():** Diese Methode wird aufgerufen, wenn Daten über den seriellen Port empfangen werden. Sie liest alle verfügbaren Daten aus dem seriellen Port, konvertiert sie in einen QString und zeigt sie im Textbrowser des GUI an. Die empfangenen Nachrichten werden blau angezeigt.

```

//Receive Message
void MainWindow::receiveMessage()
{
    QByteArray dataBA = serialPort.readAll();
    QString data(dataBA);

    ui->textBrowser->setTextColor(Qt::blue); // Receieved message's color is blue.
    ui->textBrowser->append(data);
}

```

Abbildung 40: mainwindows.cpp -Methode ,receiveMessage' der Klasse MainWindow

- **on_pushButton_5_clicked():** Diese Methode wird ausgeführt, wenn der Benutzer auf den pushButton_5 klickt. Sie liest dem Text aus einer Line-Edit Widget der vom Benutzer eingegeben wurde und sendet ihn über den seriellen Port. Der gesendete Text wird grün im Textbrowser angezeigt.

```
//Button Send
void MainWindow::on_pushButton_5_clicked()
{
    QString message = ui->lineEdit->text();
    ui->textBrowser->setTextColor(Qt::darkGreen); // Color of message to send is green.
    ui->textBrowser->append(message);
    serialPort.write(message.toUtf8());
}
```

Abbildung 41: mainwindows.cpp -Methode ,on_pushButton_5_clicked‘ der Klasse MainWindow

- **on_pushButton_2_clicked():** Diese Methode wird ausgeführt, wenn der Benutzer auf den pushButton_2 klickt. Sie schließt den seriellen Port und zeigt eine Benachrichtigung an, dass die Verbindung getrennt wurde. Die Nachricht wird rot im Textbrowser angezeigt.

```
// Button Disconnect
void MainWindow::on_pushButton_2_clicked()
{
    serialPort.close();
    QString message = "DISCONNECTED !!";
    ui->textBrowser->setTextColor(Qt::red); // Color of message to send is green.
    ui->textBrowser->append(message);
}
```

Abbildung 42: mainwindows.cpp -Methode ,on_pushButton_2_clicked‘ der Klasse MainWindow

- **on_pushButton_6_clicked():** Diese Methode wird ausgeführt, wenn der Benutzer auf den pushButton_6 klickt. Sie löscht alle vorhandenen Einträge in einer ComboBox und fügt dann alle verfügbaren seriellen Anschlüsse hinzu, die auf dem Computer gefunden wurden. Dies ermöglicht es dem Benutzer, den Port auszuwählen, mit dem die Kommunikation stattfinden soll.

```
// Button Refresh Ports
void MainWindow::on_pushButton_6_clicked()
{
    ui->comboBox->clear();
    QList<QSerialPortInfo> ports = info.availablePorts();
    QList<QString> stringPorts;
    for(int i = 0 ; i < ports.size() ; i++){
        stringPorts.append(ports.at(i).portName());
    }
    ui->comboBox->addItem(stringPorts);
}
```

Abbildung 43: Methode ,on_pushButton_6_clicked‘ der Klasse MainWindow

- **on_pushButton_4_clicked():** Diese Methode wird ausgeführt, wenn der Benutzer auf einen weiteren Push-Button klickt (normalerweise pushButton_4 genannt). Sie löscht alle Nachrichten im Textbrowser.

```
// Button of Clear
void MainWindow::on_pushButton_4_clicked()
{
    ui->textBrowser->clear();
}
```

Abbildung 44: Methode ,on_pushButton_4_clicked‘ der Klasse MainWindow

- **on_pushButton_3_clicked():** Diese Methode wird ausgeführt, wenn der Benutzer auf den pushButton_3 klickt. Sie öffnet einen Dateidialog, der es dem Benutzer ermöglicht, eine Bilddatei auszuwählen (z.B. PNG, JPG usw.). Die ausgewählte Bilddatei wird dann geladen, in das RGB-Format konvertiert und über den seriellen Port gesendet. Die Größe des Bildes wird im Textbrowser angezeigt.

```
// Button Download Image
void MainWindow::on_pushButton_3_clicked()
{
    QString fileName = QFileDialog::getOpenFileName(this, "Select an image", "", "Images (*.png *.jpg *.jpeg)");
    QImage img(fileName);

    if (img.isNull() || fileName.isEmpty()) {
        qDebug() << "Error by the image load.";
    }
    else
    {
        displayImage(img);

        img = img.scaled(96,96);

        img = img.convertToFormat(QImage::Format_RGB888);

        // image dimensions
        //int width = img.width();
        //int height = img.height();

        // convert the image in QVector Array
        QVector<uchar> image_array;

        for (int y = 0; y < 96; ++y) {
            for (int x = 0; x < 96; ++x) {
                QRgb pixel = img.pixel(x, y);
                image_array.append(qRed(pixel));
                image_array.append(qGreen(pixel));
                image_array.append(qBlue(pixel));
            }
        }
        qDebug() << "Array Size:" << image_array.size();
        QString message = QString::number(image_array.size());
        ui->textBrowser->setTextColor(Qt::darkGreen); // Color of message to send is green.
        ui->textBrowser->append(message);

        serialPort.write(reinterpret_cast<const char*>(image_array.data()), image_array.size());
    }
}
```

Abbildung 45: Methode ,on_pushButton_3_clicked‘ der Klasse MainWindow

- **displayImage(const QImage &img):** Diese Methode zeigt das geladene Bild in einem QLabel in der GUI an. Das Bild wird skaliert, um es im Label anzuzeigen.

```
void MainWindow::displayImage(const QImage &img)
{
    // show image in QLabel
    ui->labelImage->setPixmap(QPixmap::fromImage(img));
    ui->labelImage->setScaledContents(true);
}
```

Abbildung 46: Methode ‚displayImage‘ der Klasse MainWindow

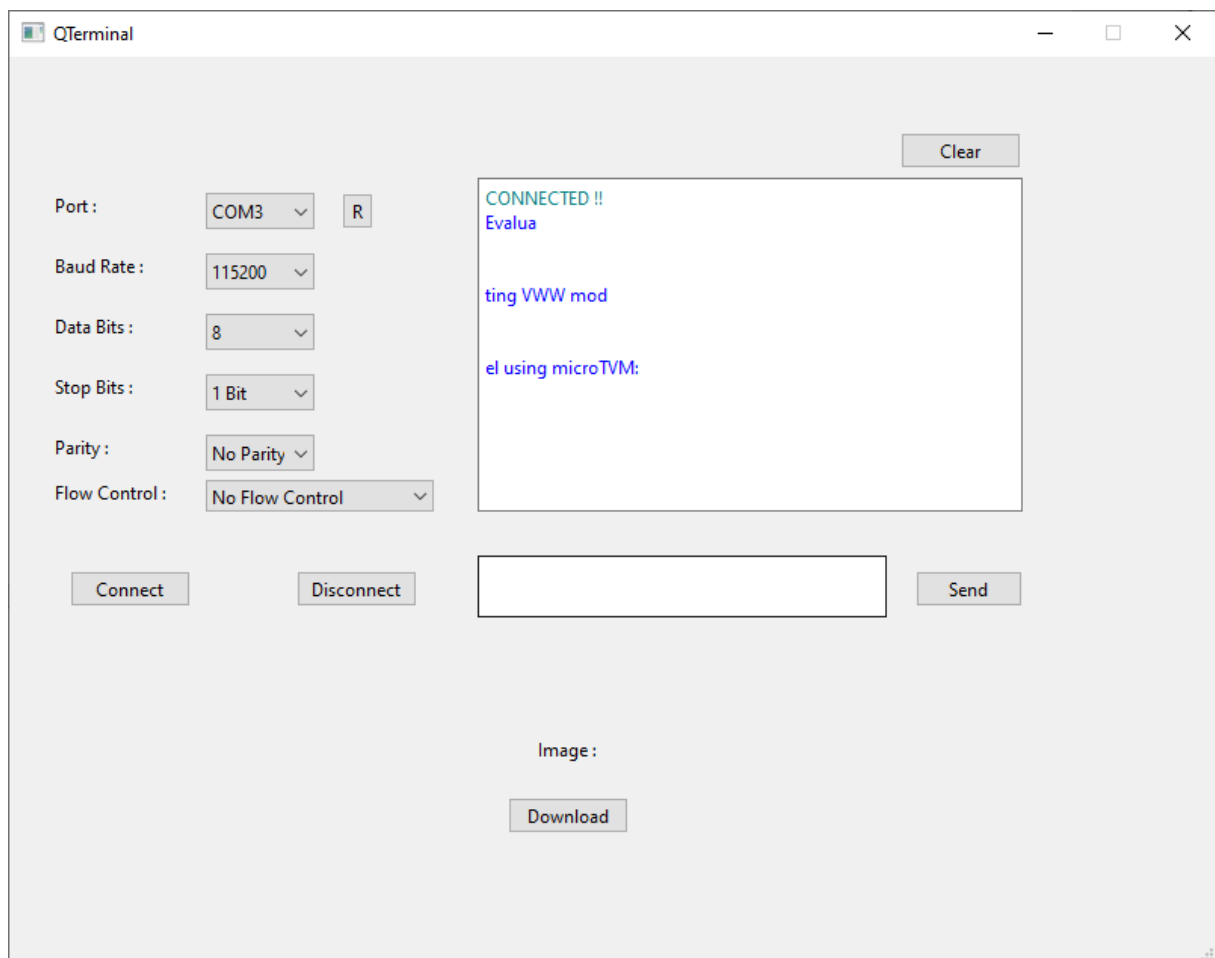


Abbildung 47: Darstellung der erstellten Benutzeroberfläche

Expression	Type	Value
▼ [0..99]		[27648]
(x)= bufferData[0]	uint8_t	183 '.'
(x)= bufferData[1]	uint8_t	188 '¼'
(x)= bufferData[2]	uint8_t	191 '¿'
(x)= bufferData[3]	uint8_t	183 '.'
(x)= bufferData[4]	uint8_t	188 '¼'
(x)= bufferData[5]	uint8_t	191 '¿'
(x)= bufferData[6]	uint8_t	183 '.'
(x)= bufferData[7]	uint8_t	188 '¼'
(x)= bufferData[8]	uint8_t	191 '¿'
(x)= bufferData[9]	uint8_t	184 'ä'
(x)= bufferData[10]	uint8_t	188 '¼'
(x)= bufferData[11]	uint8_t	189 '½'
(x)= bufferData[12]	uint8_t	184 'ä'
(x)= bufferData[13]	uint8_t	188 '¼'
(x)= bufferData[14]	uint8_t	189 '½'
(x)= bufferData[15]	uint8_t	184 'ä'

Abbildung 50: Live Expression - STM32Cube IDE

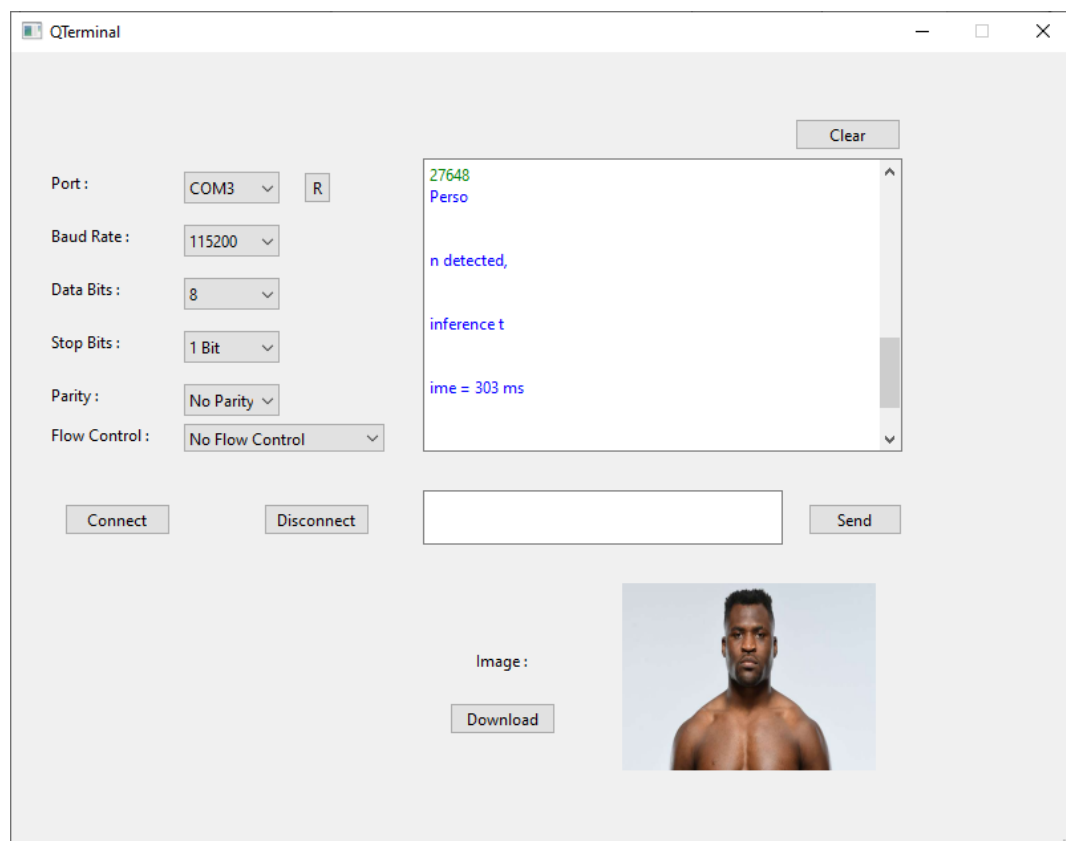


Abbildung 51: Bild 1

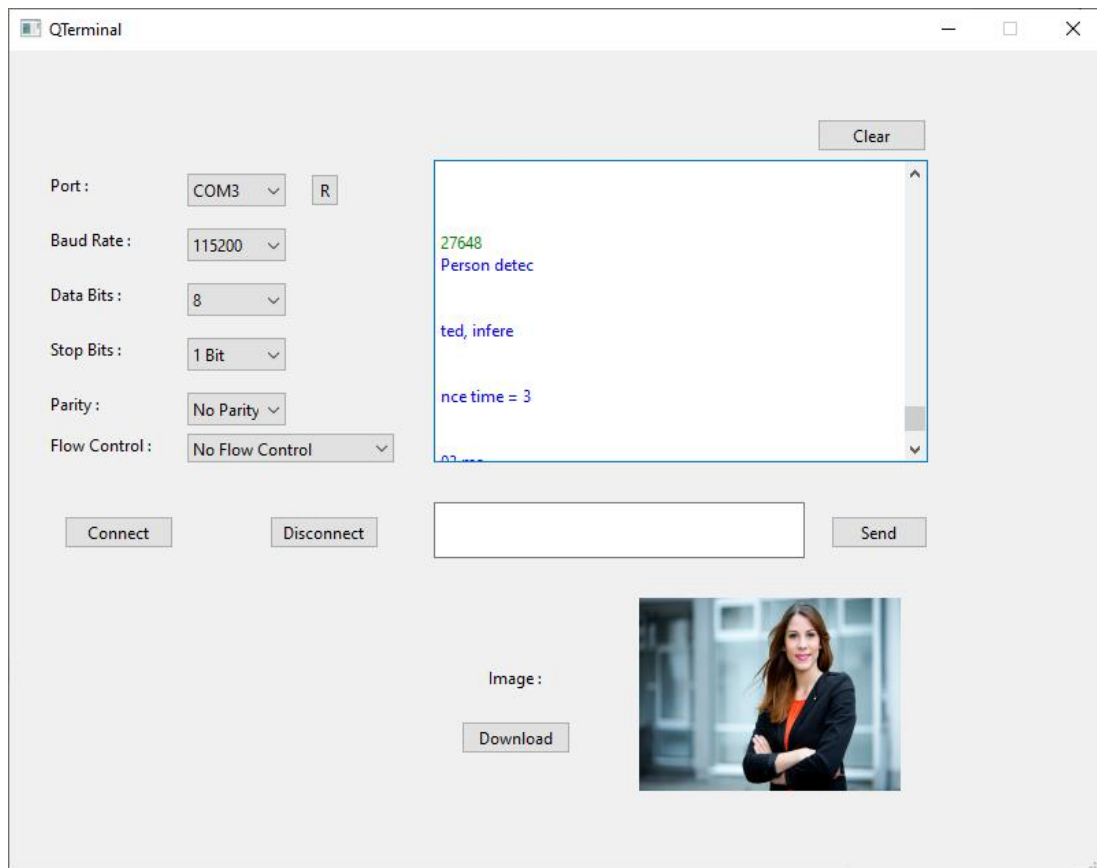


Abbildung 52: Bild 6

Die Erkennung von Personen in Bildern stellt eine wichtige Anwendung des Visual Wake Word Tensor Flow Lite Models (TFLM) dar. Um das TFLM effektiv zu nutzen, müssen die Bilder auf eine Größe von 96x96 Pixeln mit 3 RGB-Kanälen skaliert werden, da das Modell diese Eingabe erwartet.

Wichtig ist es, das Seitenverhältnis beizubehalten, da das TFLM nur Bilder mit korrektem Seitenverhältnis verarbeiten kann. Dies kann jedoch eine Herausforderung sein, da das Modell nicht für Bilder mit einem anderen Seitenverhältnis ausgelegt ist.

Das RAW-Format bietet eine größere Menge an Fotodaten für die Nachbearbeitung, was für die Auswertung von Bildern von Vorteil ist. Es ermöglicht eine detaillierte Analyse und Bearbeitung der Bilddaten, um die Leistung des Modells zu verbessern.

Bei der Auswertung von Bildern ist festzustellen, dass das TFLM gut funktioniert, wenn mindestens die Hälfte des Oberkörpers einer Person im Bild zu sehen ist (Abbildung 51: Bild 1 und Abbildung 52: Bild 6). Dies deutet darauf hin, dass das Modell eine ausreichende Genauigkeit aufweist, um Personen in Bildern zu erkennen, solange die relevanten Merkmale deutlich sichtbar sind.

Es ist jedoch wichtig zu beachten, dass die Nutzung des TFLM auch Rechenressourcen erfordert. Insgesamt bietet die Auswertung von Bildern mithilfe des TFLM eine effektive Methode zur Erkennung von Personen in Bildern und bietet Potenzial für vielfältige Anwendungen im Bereich der visuellen Erkennung und Analyse.

Fazit

Künstliche Intelligenz hat bereits einen erheblichen Einfluss auf unser tägliches Leben und wird in Zukunft voraussichtlich noch mehr an Bedeutung gewinnen. Sie bietet zahlreiche Einsatzmöglichkeiten in verschiedenen Branchen und Anwendungen, die uns helfen, Herausforderungen zu bewältigen. Es ist wichtig, dass die Entwicklung der KI im Einklang mit den gesellschaftlichen Werten und Zielen erfolgt, um die Gesellschaft nicht zu gefährden. Der Fokus sollte auf praktischen Anwendungsfällen liegen, um die Vorteile der KI im Alltag nutzbar zu machen.

Ein Beispiel für eine KI-Anwendung ist die Bilderkennung mit dem NUCLEO L4R5ZI-Board von ST, die in dieser Arbeit durchgeführt wurde. Es wurde ein Algorithmus verwendet, der anhand eines Bildes erkennt, ob eine Person auf dem Bild zu sehen ist. Dies ist nur eine der vielen Möglichkeiten, in denen KI in Zukunft eine wichtige Rolle in unserem täglichen Leben spielen wird. Das Projekt wurde mit vorentwickelten Modellen durchgeführt und erfolgreich in von STM bereitgestellte Entwicklungsumgebung integriert.

Quelle und Literaturverzeichnis

- [1] <https://studyflix.de/informatik/neuronale-netze-4297>
- [2] <https://datasolut.com/was-ist-deep-learning/>
- [3] <https://studyflix.de/informatik/machine-learning-4356>
- [4] https://colab.research.google.com/github/apache/tvm-site/blob/asf-site/docs/_downloads/a74627f44186b95116fe0ed6f77e3b99/micro_custom_ide.ipynb#scrollTo=GAbABcftEMZ
- [5] Géron, Aurélien. **Praxiseinstieg Machine Learning mit Scikit-Learn, Keras und TensorFlow: Konzepte, Tools und Techniken für intelligente Systeme**. O'Reilly Verlag, 2019.
- [6] Remzi, Karaarslan, Projektarbeit: „Edge Computing für KI-Anwendung“ betreut von Prof. Dr. -Ing. Michael Athanassios Karagounis

Abbildungsverzeichnis

Abbildung 1: Machine Learning Anwendungsbereiche [1]	5
Abbildung 2: Ein Bild von Hund mit dem richtigen Ergebnis eingeben (Klassifikation= Hund und Regression= 40 in kg) [1]	6
Abbildung 3: nur verschiedene Tierbilder, ohne Beschriftung (Clustering = Gruppe von Fisch, Gruppe von Hunden... ..	6
Abbildung 4: das manuelle Beschriften von Bildern mit der passenden Tierart [1]	7
Abbildung 5: verstärkendes Lernen [1]	7
Abbildung 6: Aufbau Neuronaler Netze [1]	9
Abbildung 7: Deep Learning, Teil der künstlichen Intelligenz [2]	11
Abbildung 8: Machine Learning vs Deep Learning [3]	12
Abbildung 9: NUCLEO-L4R5ZI - STM32 Nucleo Board 2MB 640KB	14
Abbildung 10: Google Colab Terminal	15
Abbildung 11: Bild mit Person [4] Abbildung 12: Bild ohne Person [4]	19
Abbildung 13: Hierarchie der generierten Datei model.tar	21
Abbildung 14: STM32 CubeIDE Board Selector	22
Abbildung 15: Projektname	23
Abbildung 16: UART-Einstellungen	23
Abbildung 17: Hello World. Ausgabe im Terminalprogramm PuTTY	24
Abbildung 18: Remote Connection	25
Abbildung 19: Serial Port Connection	25
Abbildung 20: Paths and Symbols	26
Abbildung 21: Projektauflistung	26
Abbildung 22: Settings - include Paths	27
Abbildung 23: Ordnerstruktur	28
Abbildung 24: Code in der Endloschleife	31
Abbildung 25: Neues Projekt auf Qt Creator	32
Abbildung 26: Projektname	33
Abbildung 27: Build-System	33
Abbildung 28: Kits-Auswahl	34

Abbildung 29: Zusammenfassung der Projekt-Erstellung	34
Abbildung 30: Qt-Verwaltung (Maintenance Tool)	35
Abbildung 31: main.cpp	36
Abbildung 32: Editierbare Darstellung der Datei mainwindow.ui	37
Abbildung 33: Inhalt der pro-Datei	37
Abbildung 34: mainwindow.h	38
Abbildung 35: mainwindow.cpp - Header File und Pakete laden	39
Abbildung 36: mainwindow.cpp- Default Konstruktor der Klasse MainWindow	39
Abbildung 37: mainwindow.cpp - Destruktor	40
Abbildung 38: Methode 'on_pushButton_clicked' der Klasse MainWindow	41
Abbildung 39: Methode 'on_pushButton_clicked' der Klasse MainWindow	42
Abbildung 40: mainwindows.cpp -Methode ,receiveMessage' der Klasse MainWindow	42
Abbildung 41: mainwindows.cpp -Methode ,on_pushButton_5_clicked ' der Klasse MainWindow	43
Abbildung 42: mainwindows.cpp -Methode ,on_pushButton_2_clicked ' der Klasse MainWindow	43
Abbildung 43: Methode ,on_pushButton_6_clicked ' der Klasse MainWindow	43
Abbildung 44: Methode ,on_pushButton_4_clicked ' der Klasse MainWindow	44
Abbildung 45: Methode ,on_pushButton_3_clicked ' der Klasse MainWindow	44
Abbildung 46: Methode ,displayImage' der Klasse MainWindow	45
Abbildung 47: Darstellung der erstellten Benutzeroberfläche	45
Abbildung 48: Von Google Colab im model.tar erstellende Header File	46
Abbildung 49: Von unserem QT-App gesendete Datei	46
Abbildung 50: Live Expression - STM32Cube IDE	47
Abbildung 51: Bild 1	47
Abbildung 57: Bild 6	48